



CIFRADO SIMÉTRICO

EN BLOQUE





Algoritmo
de cifrado



Algoritmo
de descifrado



Existen dos tipos de cifrado simétrico

FLUJO

Va cifrando bit a bit
sobre la marcha

BLOQUE

Corta el mensaje en
trozos de tamaño fijo y
cifra cada trozo





Algoritmo
de cifrado



Algoritmo
de descifrado



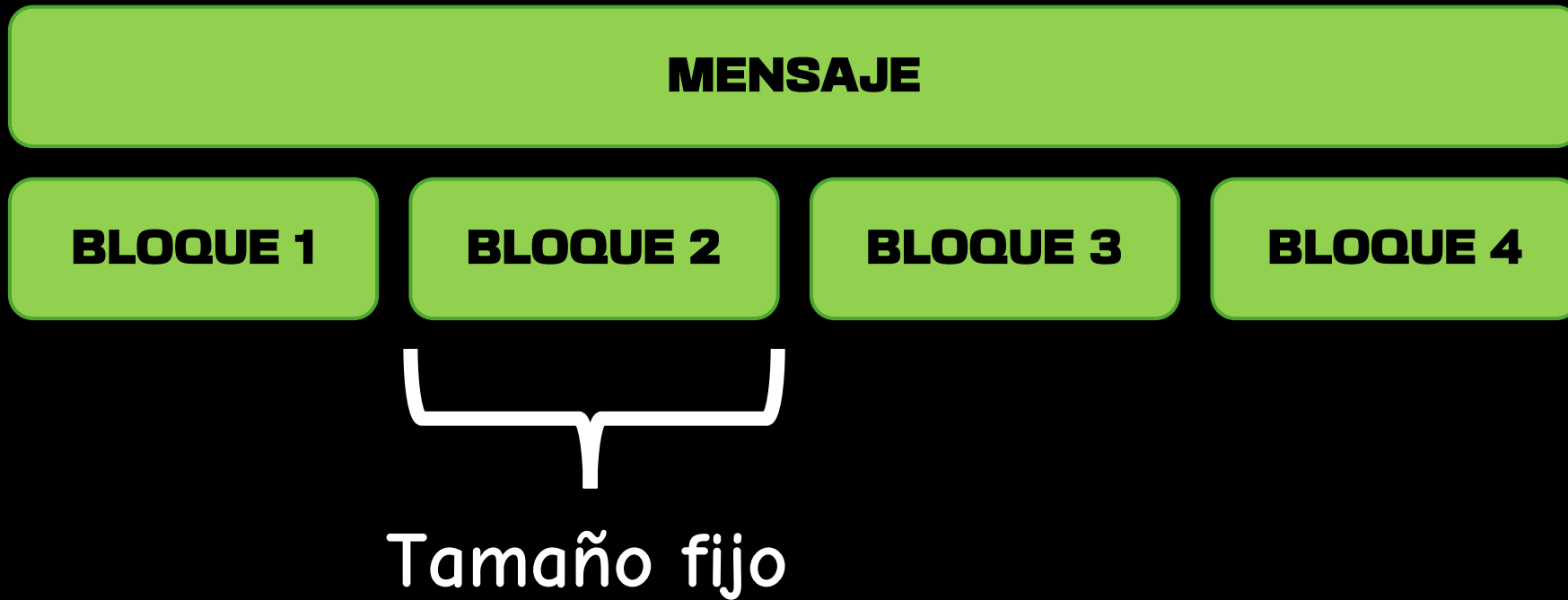
BLOQUE

Corta el mensaje en trozos de tamaño fijo
y cifra cada trozo

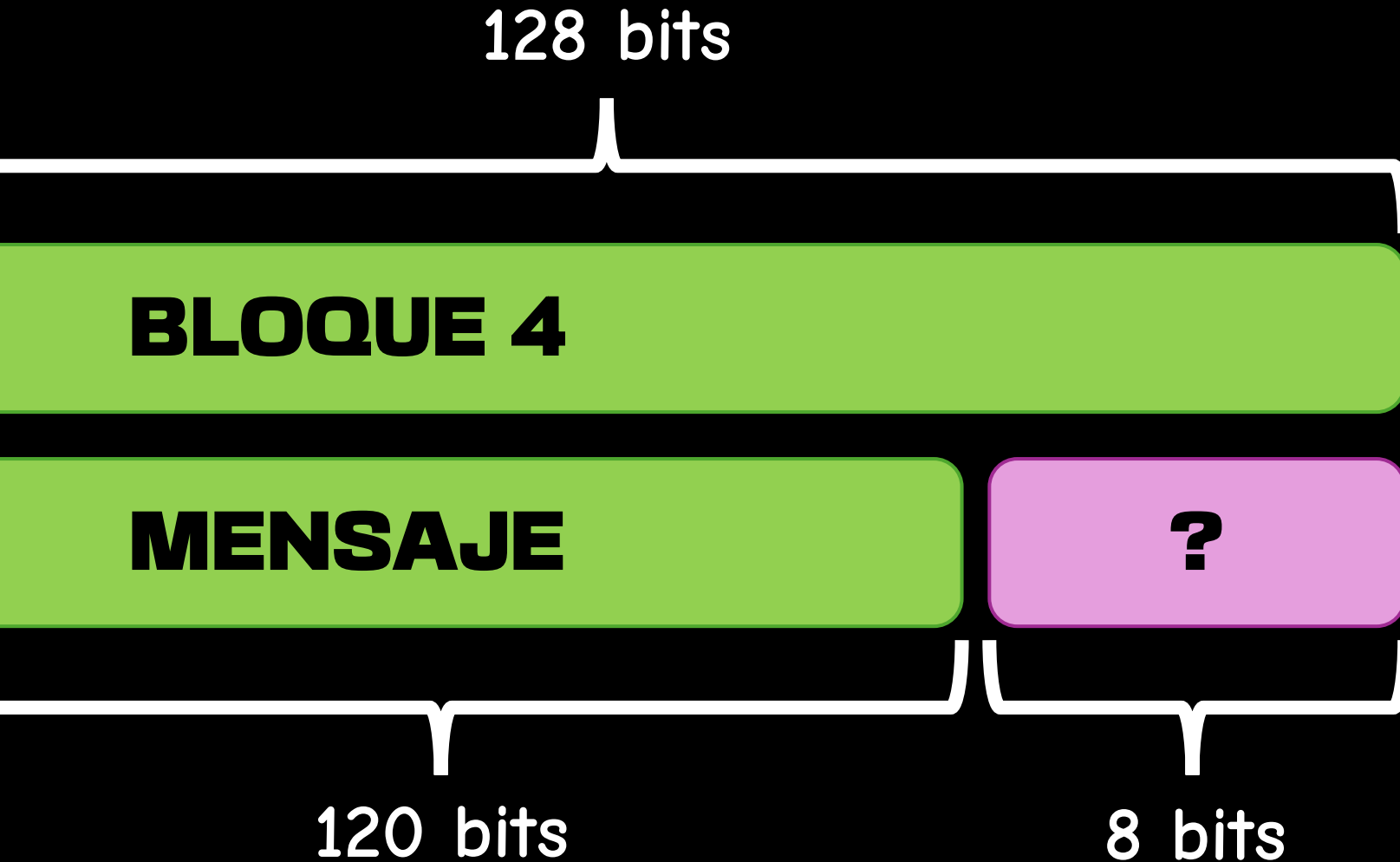
Datos estáticos que ya conocemos



Cifrado en bloque



Cifrado en bloque



Cifrado en bloque

BLOQUE 4

MENSAJE

?

PADDING



Cifrado en bloque

**MENSAJE
(BLOQUE 1)**

XOR

KEY

=

**CIFRADO
(BLOQUE 1)**

**MENSAJE
(BLOQUE 2)**

XOR

KEY

=

**CIFRADO
(BLOQUE 2)**

**MENSAJE
(BLOQUE 3)**

XOR

KEY

=

**CIFRADO
(BLOQUE 3)**

**MENSAJE
(BLOQUE 4)**

XOR

KEY

=

**CIFRADO
(BLOQUE 4)**



Cifrado en bloque

CIFRADO

**CIFRADO
(BLOQUE 1)**

**CIFRADO
(BLOQUE 2)**

**CIFRADO
(BLOQUE 3)**

**CIFRADO
(BLOQUE 4)**



Cifrado en bloque

Pero para entenderlo bien, vamos a ver como funcionan 3 algoritmos de cifrado en bloque.

DES

3DES

AES



Cifrado en bloque

DES

Cifrado DES (el clásico)

Diseñado en 1977. Fue durante muchos años un estándar muy importante usado en sistemas bancarios y gubernamentales.

Hoy está obsoleto e inseguro por su clave corta de 56 bits.

Y para entenderlo veremos:

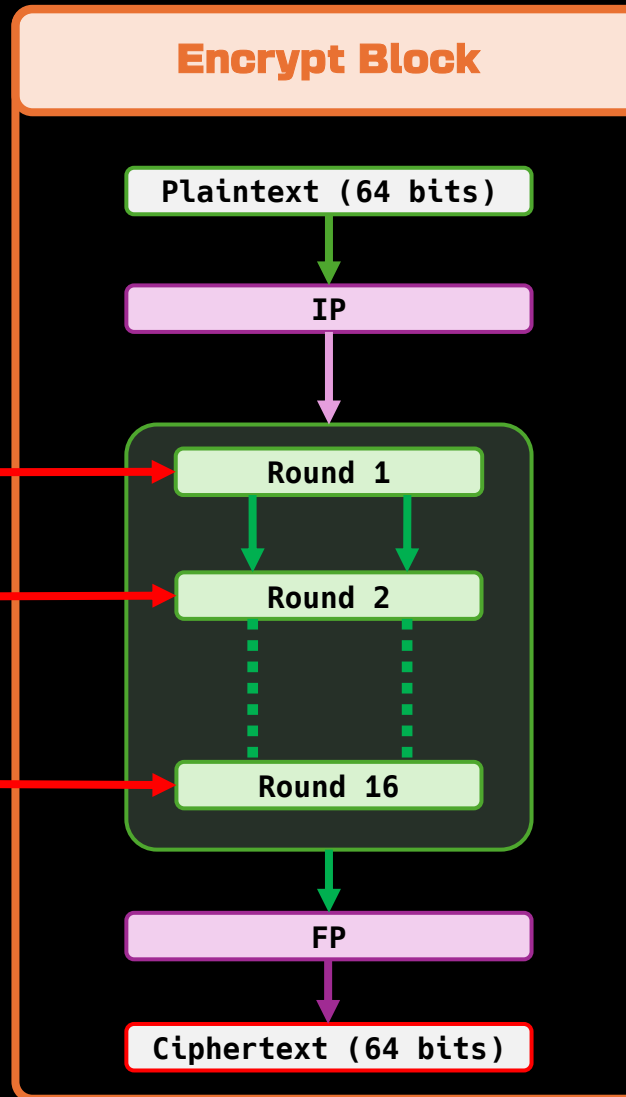
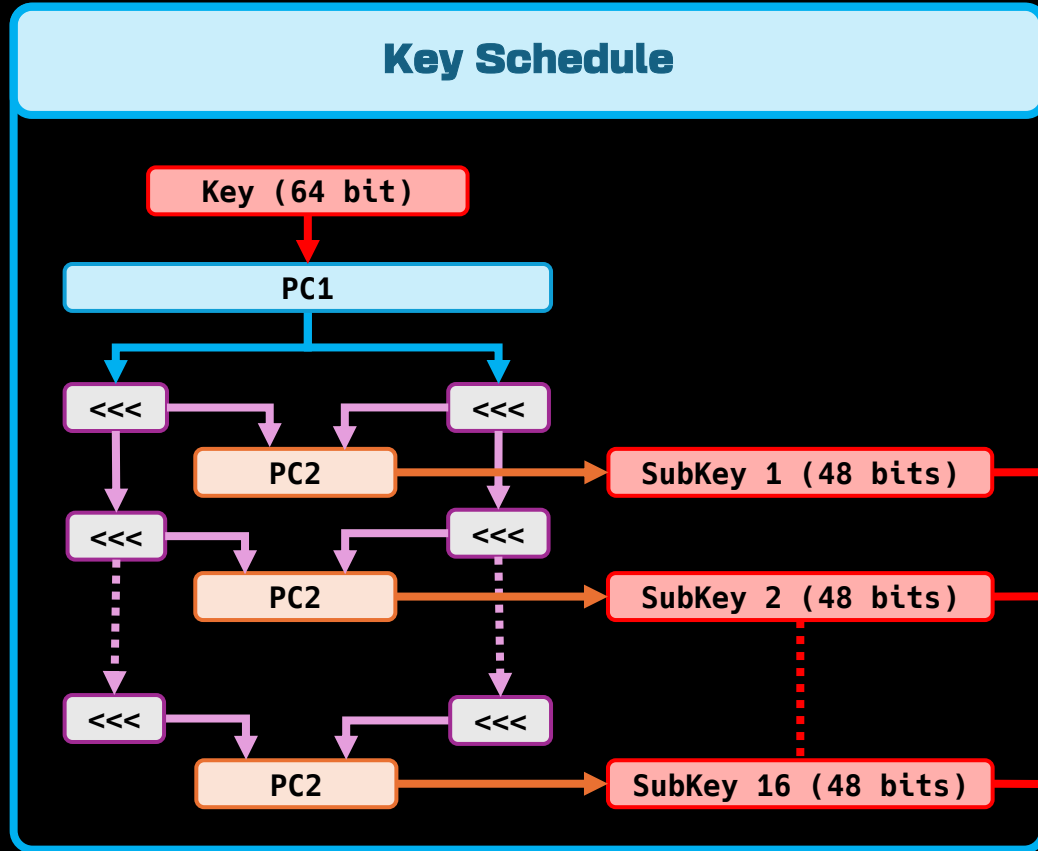
Key Schedule

Encrypt block



Cifrado en bloque

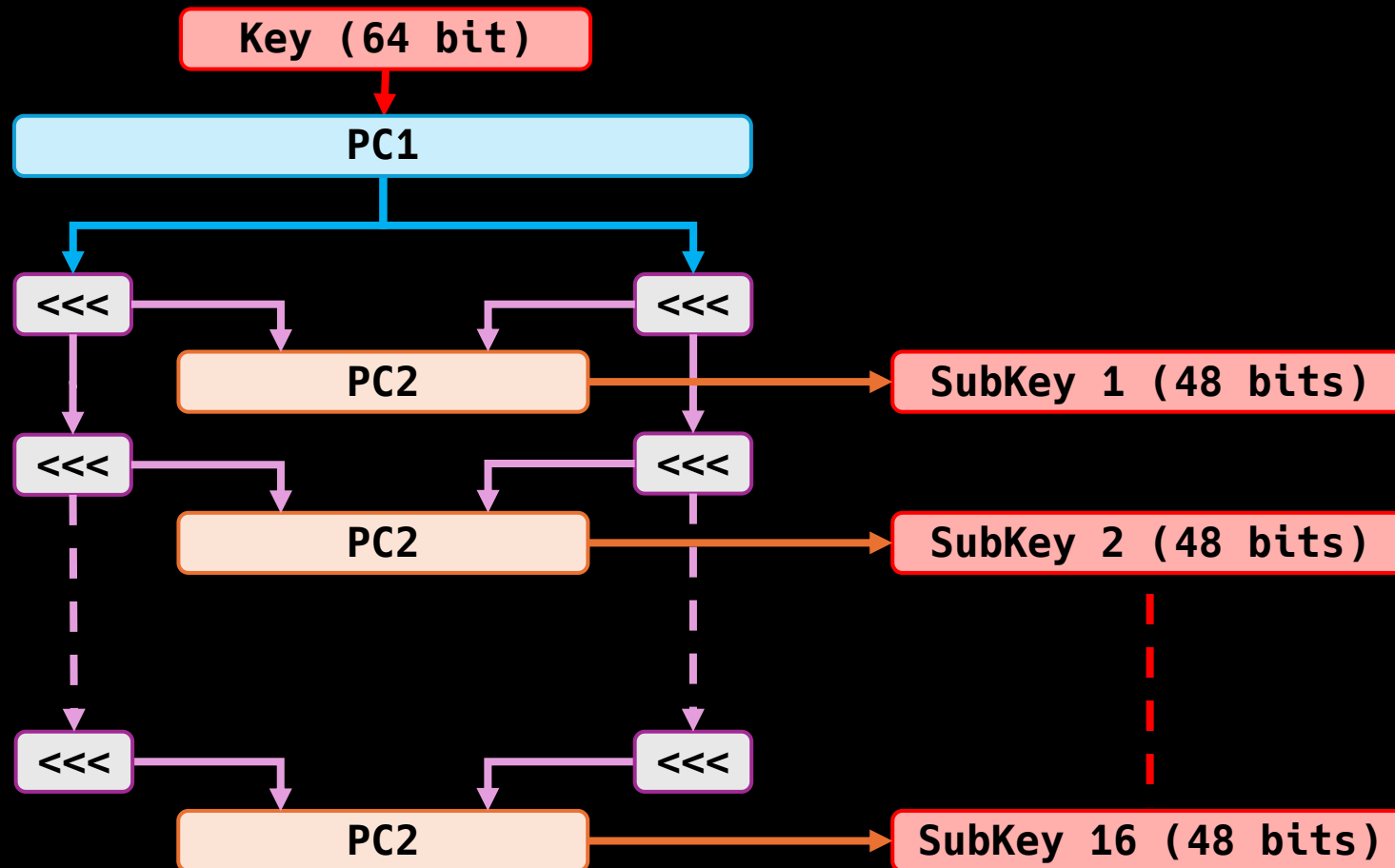
DES



Cifrado en bloque

DES

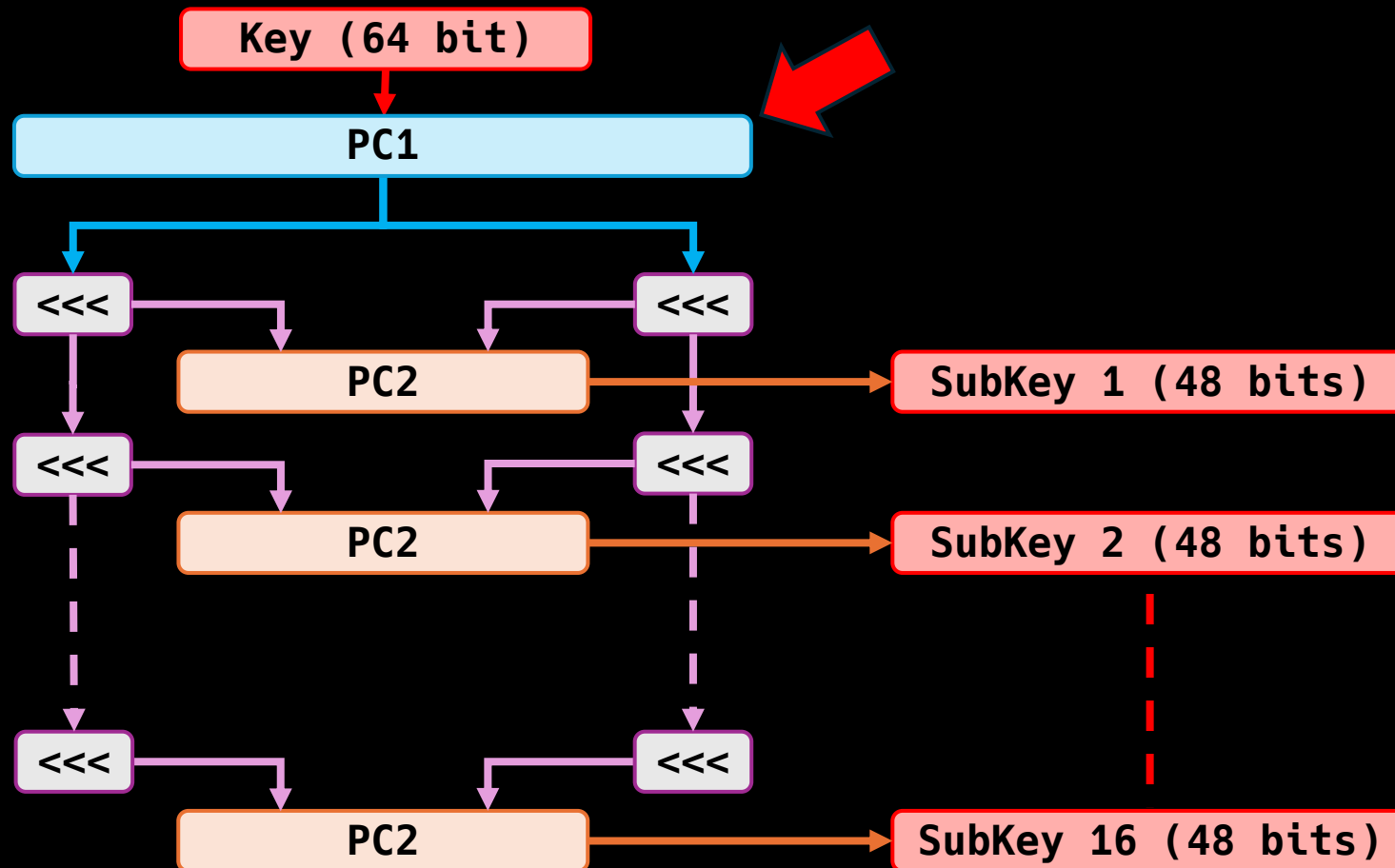
Key Schedule



Cifrado en bloque

DES

Key Schedule



Cifrado en bloque

DES

Key Schedule

PC1

Elimina los bits de paridad y reordena la clave.

Input: Key (64 bits) **Output:** permutedKey (56 bits)

```
PC1(key[64 bits]):  
  for i from 1 to 56  
    permutedKey[i] := key[PC1[i]]  
  endfor  
  return permutedKey
```

Left							Right						
57	49	41	33	25	17	9	63	55	47	39	31	23	15
1	58	50	42	34	26	18	7	62	54	46	38	30	22
10	2	59	51	43	35	27	14	6	61	53	45	37	29
19	11	3	60	52	44	36	21	13	5	28	20	12	4



Cifrado en bloque

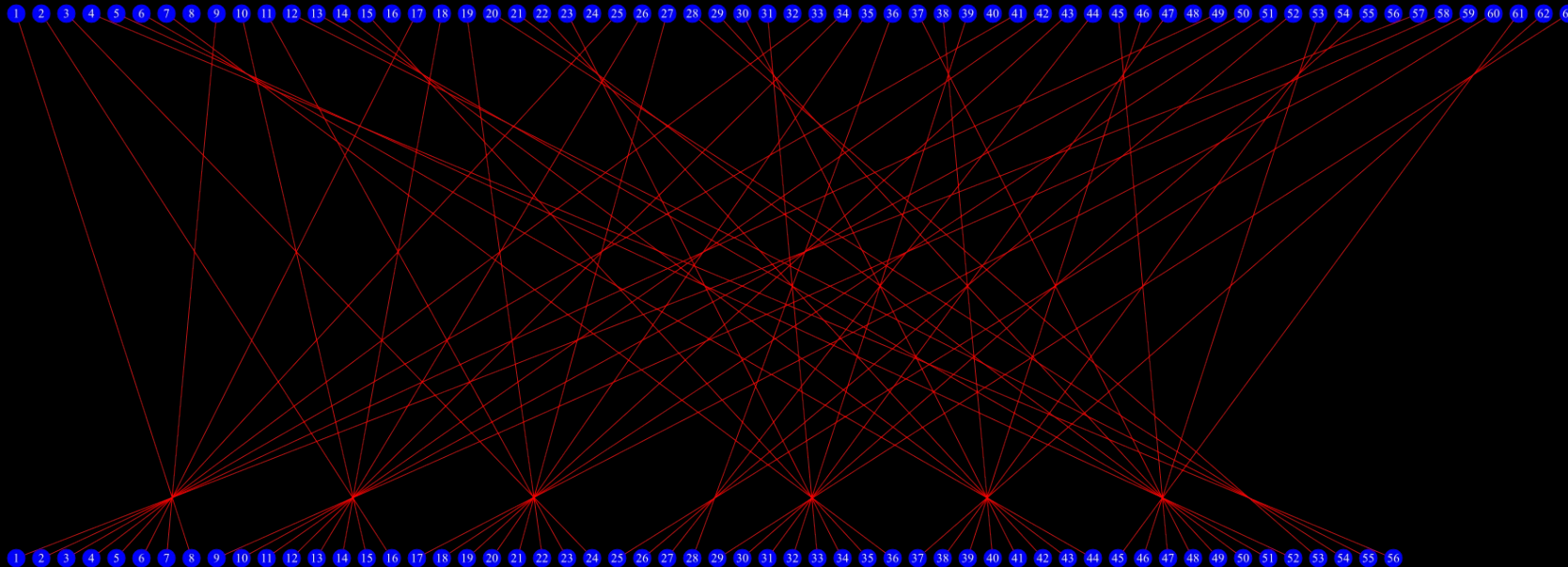
DES

Key Schedule

PC1

Elimina los bits de paridad y reordena la clave.

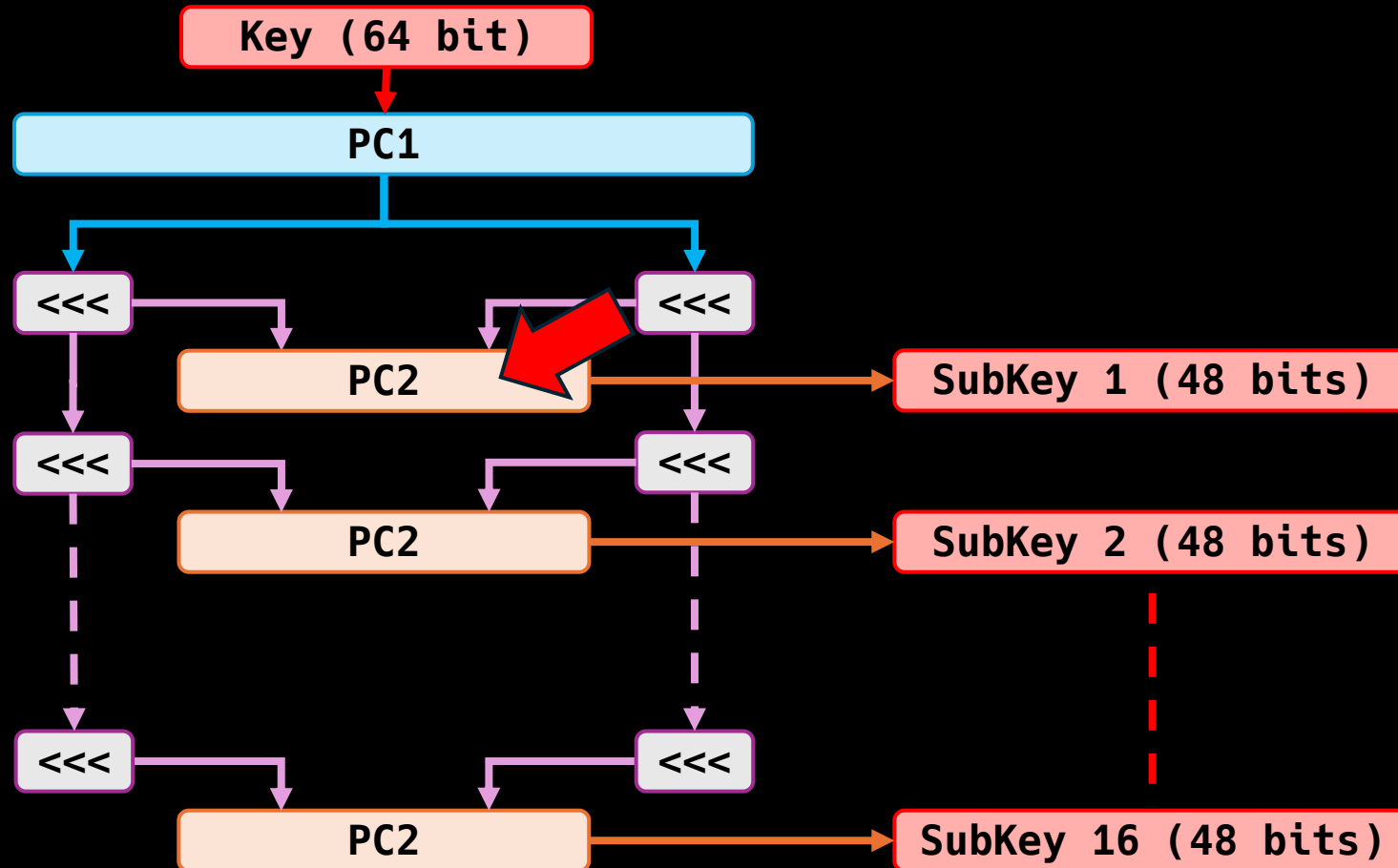
Input: Key (64 bits) **Output:** permutedKey (56 bits)



Cifrado en bloque

DES

Key Schedule



Cifrado en bloque

DES

Key Schedule

PC2

Selecciona y reordena bits para formar una subclave.

Input: combinedKey (56 bits) **Output:** subKey (48 bits)

```
PC2(combinedKey[56 bits]):  
  for i from 1 to 48  
    subkey[i] := combinedKey[PC2[i]]  
  endfor  
  return subkey
```

PC-2					
14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32



Cifrado en bloque

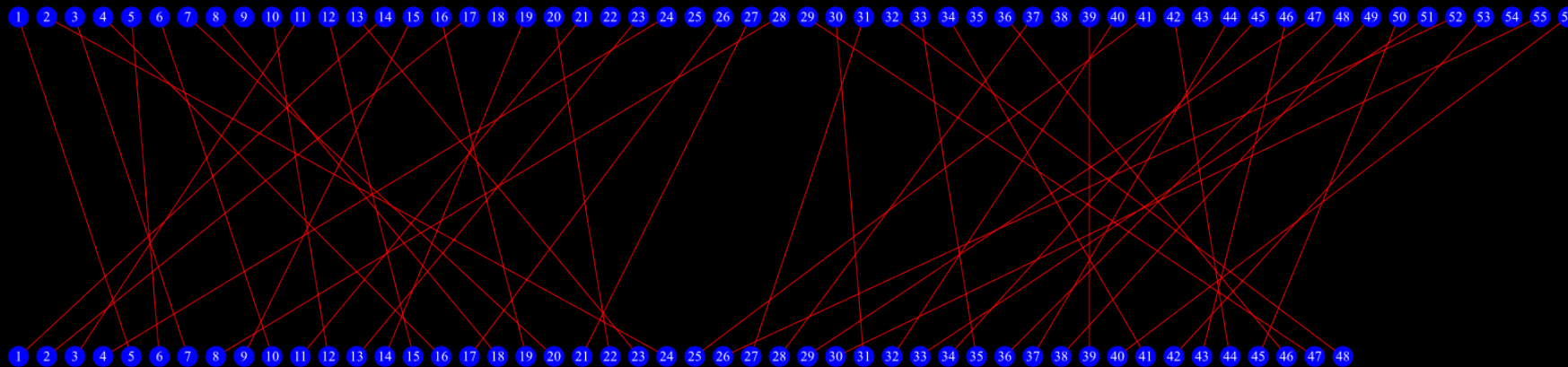
DES

Key Schedule

PC2

Selecciona y reordena bits para formar una subclave.

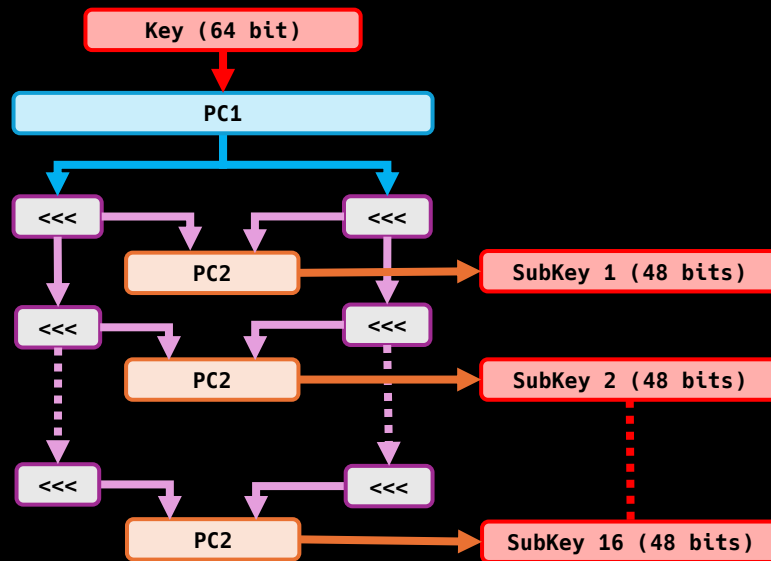
Input: combinedKey (56 bits) **Output:** subKey (48 bits)



Cifrado en bloque

DES

Key Schedule

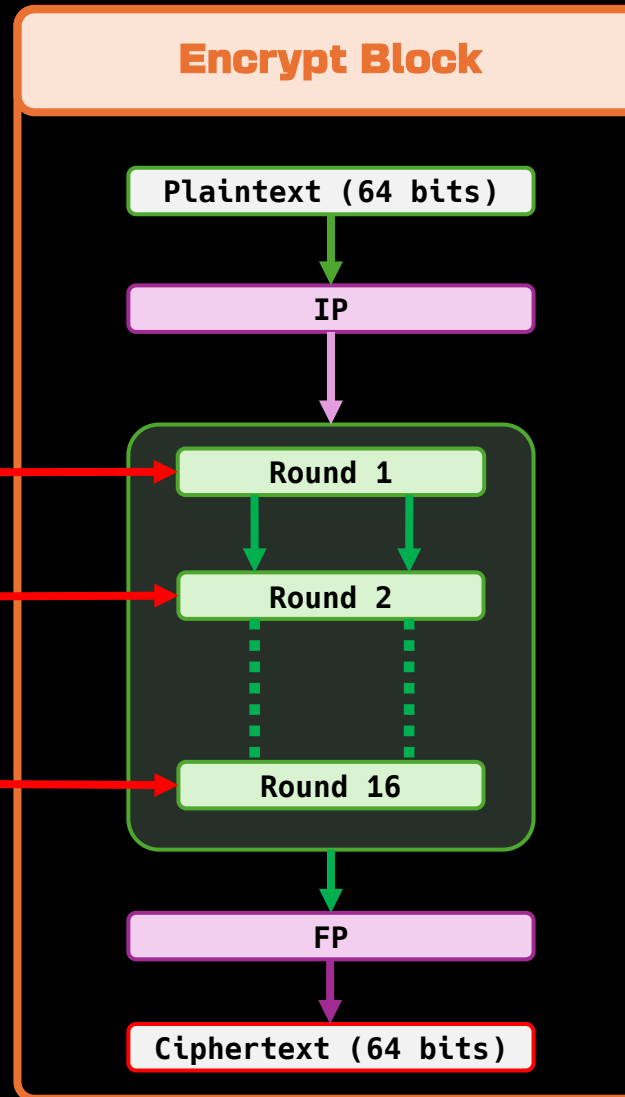
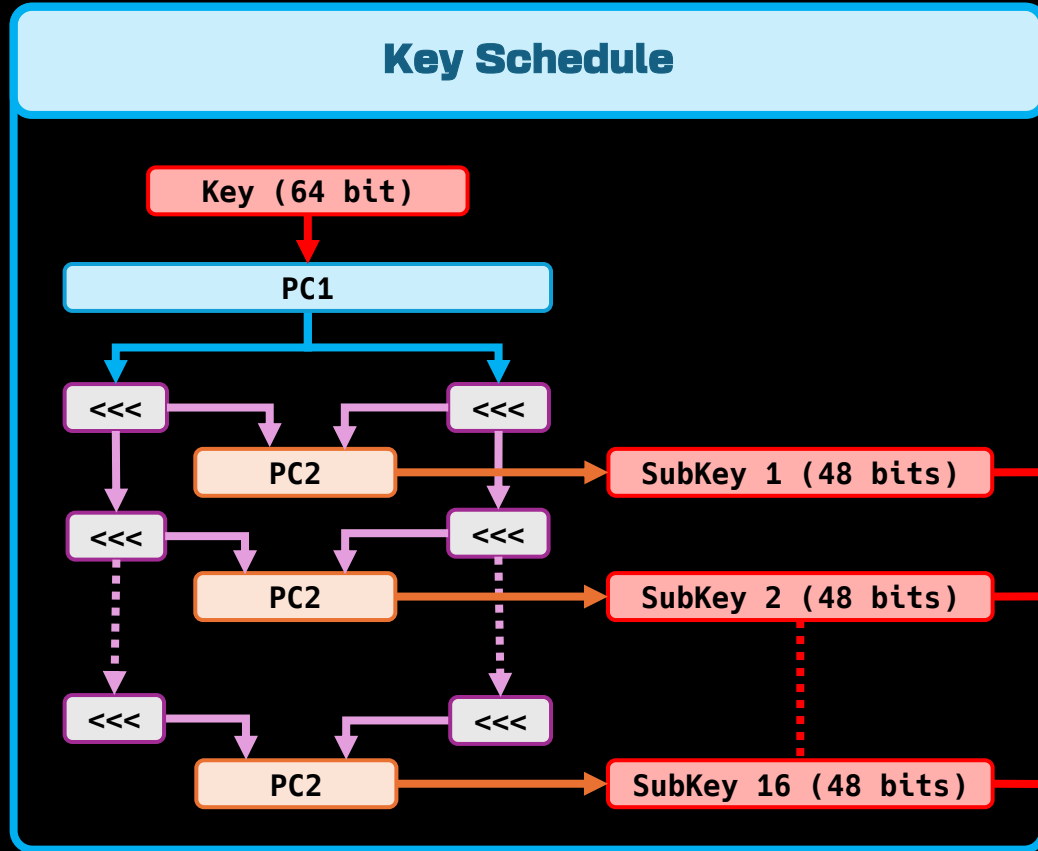


```
KeySchedulingAlgorithm(key[64 bits]):  
    key := PC1(key)  
  
    leftKey := (key rightshift 28) and 0xFFFFFFFF  
    rightKey := key and 0xFFFFFFFF  
  
    for i from 1 to 16 do  
        leftKey := leftKey leftrotate KEY_shift[i]  
        rightKey := rightKey leftrotate KEY_shift[i]  
  
        combinedKey := (leftKey leftshift 28) or rightKey  
  
        keys[i] := PC2(combinedKey)  
    endfor  
  
    return keys
```



Cifrado en bloque

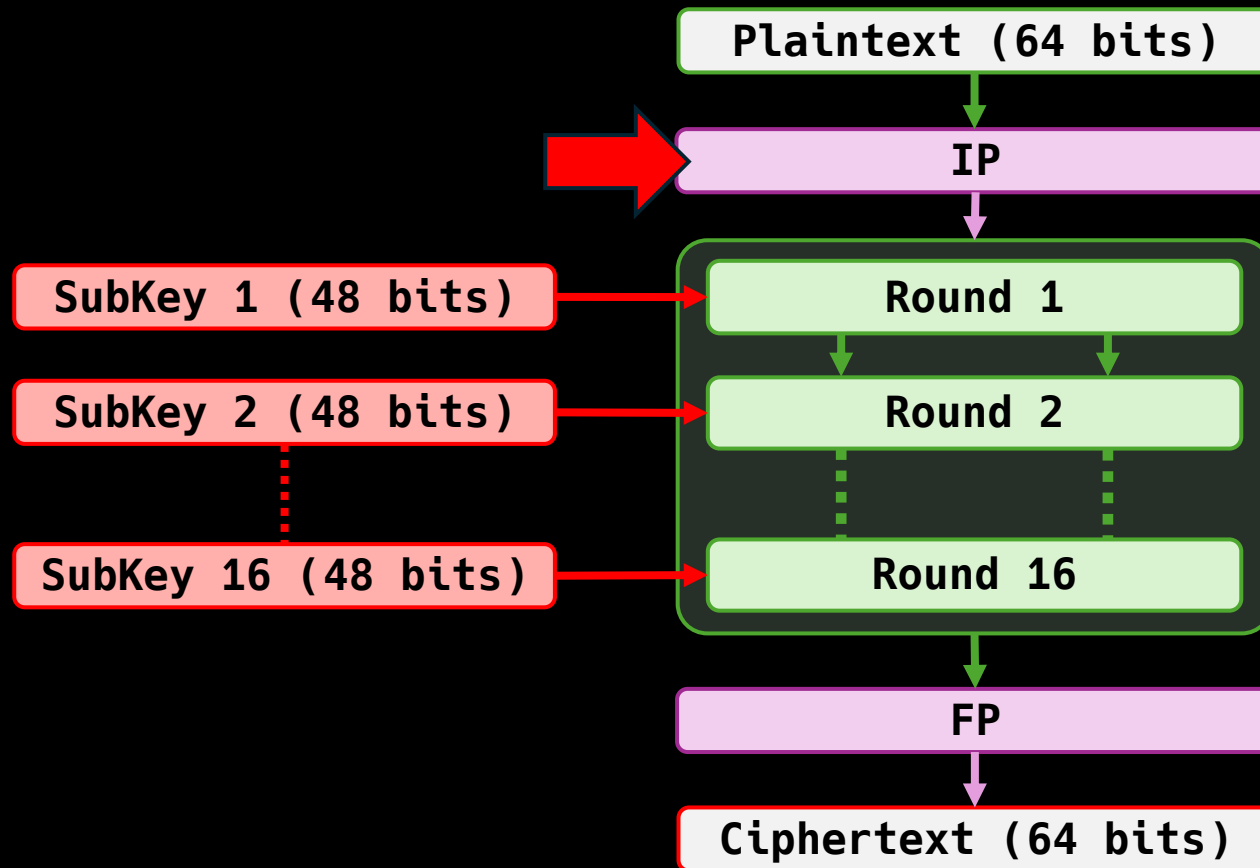
DES



Cifrado en bloque

DES

Encrypt Block



Cifrado en bloque

DES

Encrypt Block

IP

La tabla IP indica cómo se reordenan los bits originales

Input: PlainText (64 bits) **Output:** permutedBlock (64 bits)

```
IP(block[64 bits]):  
  for i from 1 to 64  
    permutedBlock[i] := block[IP[i]]  
  endfor  
  return permutedBlock
```

IP							
58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7



Cifrado en bloque

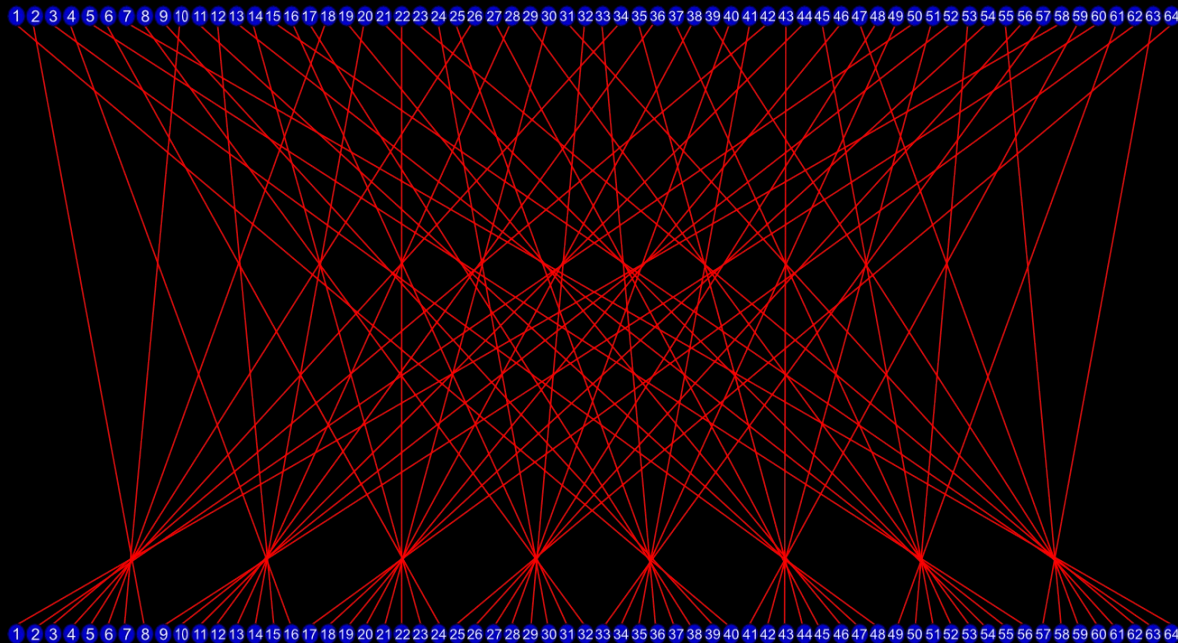
DES

Encrypt Block

IP

La tabla IP indica cómo se reordenan los bits originales

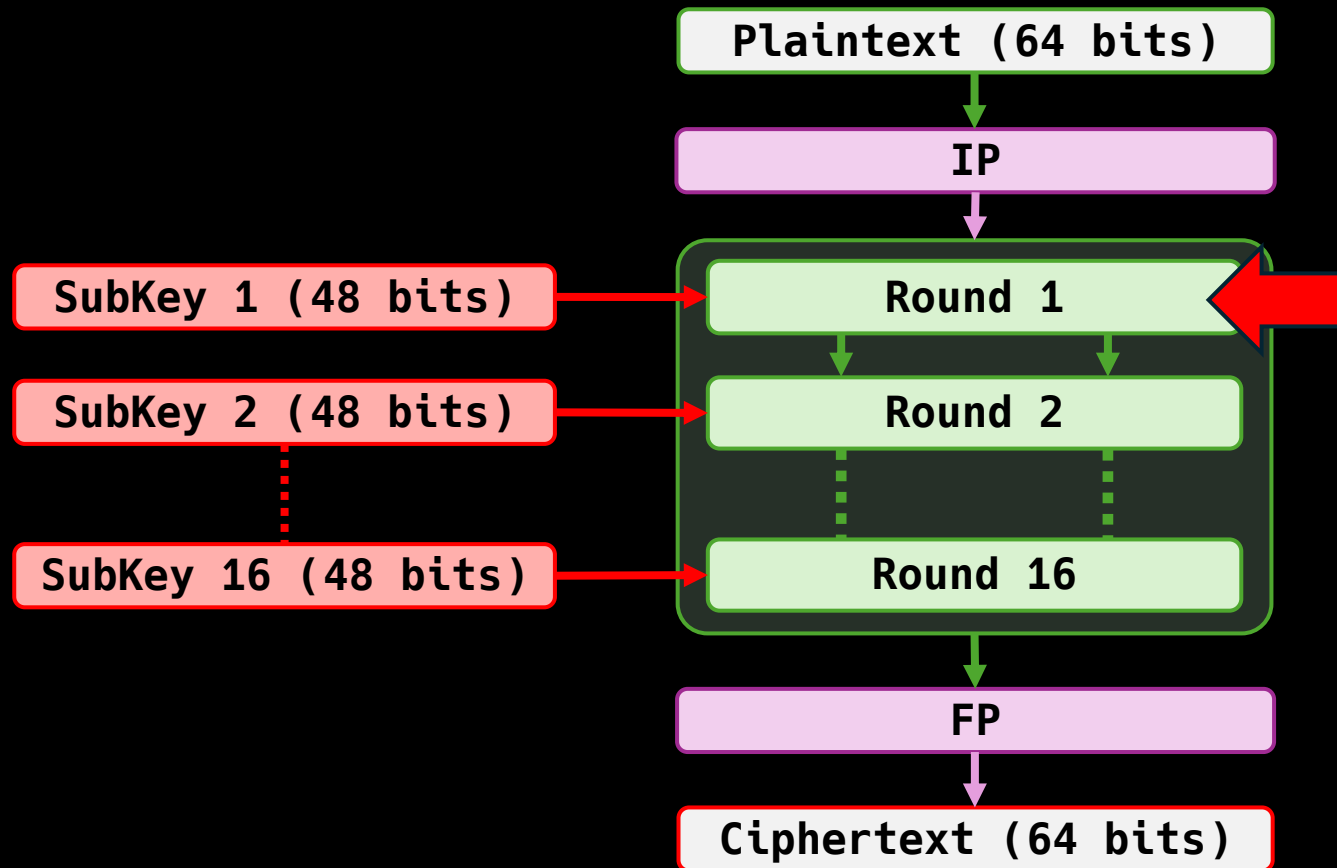
Input: PlainText (64 bits) **Output:** permutedBlock (64 bits)



Cifrado en bloque

DES

Encrypt Block



Cifrado en bloque

DES

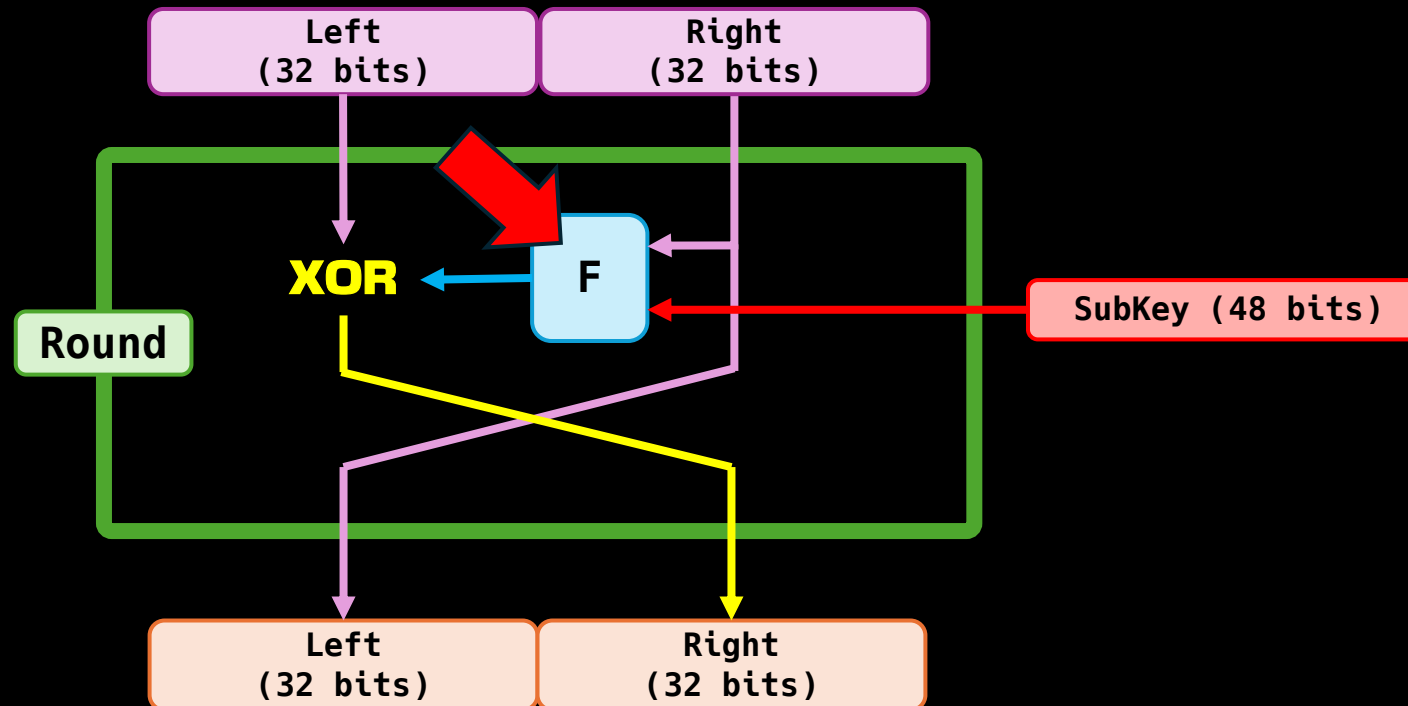
Encrypt Block

Round 1

Round 2

...

Round 16



Cifrado en bloque

DES

Encrypt Block

Round 1

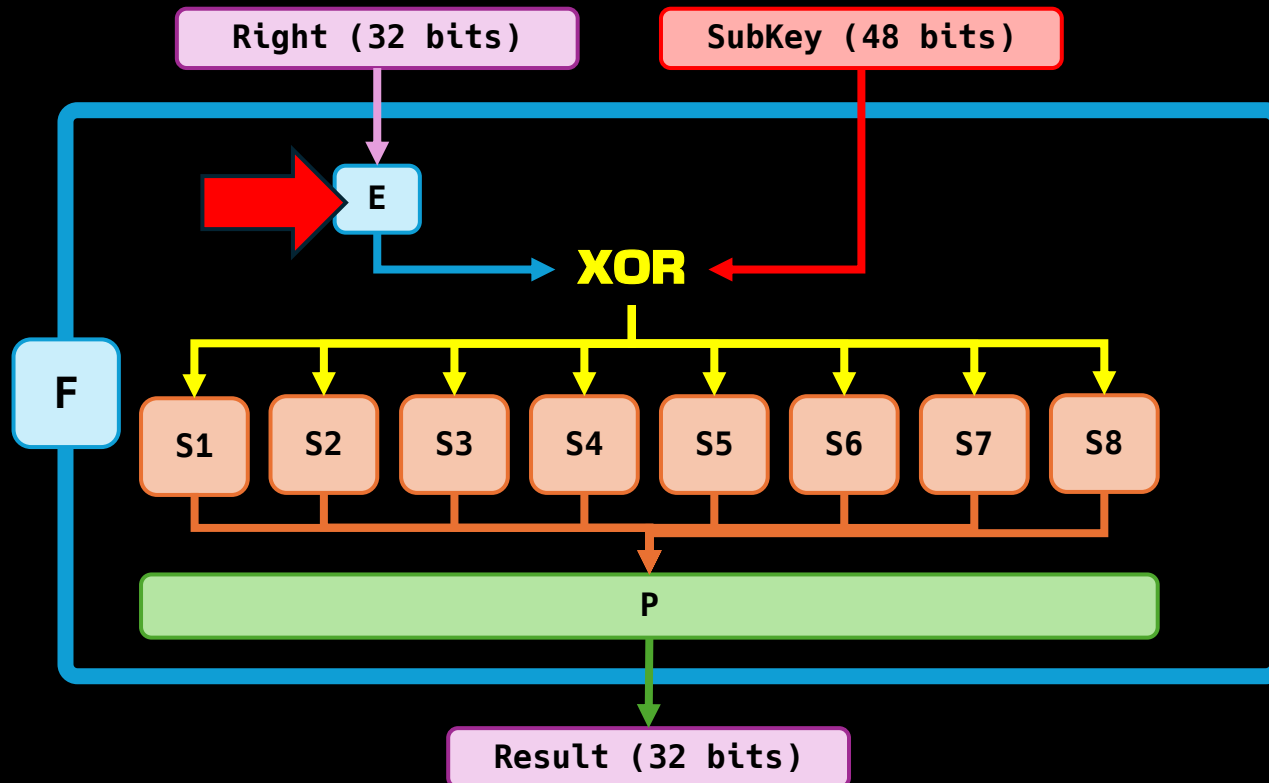
Round 2

...

Round 16



F



Cifrado en bloque

DES

Encrypt Block

Round 1

Round 2

...

Round 16



F



E

Expande la mitad derecha repitiendo algunos bits.

Input: Right (32 bits)

Output: permutedBlock (64 bits)

```
E(R[32 bits]):  
  for i from 1 to 48  
    expandedR[i] := R[E[i]]  
  endfor  
  return expandedR
```

E					
32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1



Cifrado en bloque

DES

Encrypt Block

Round 1

Round 2

...

Round 16



F

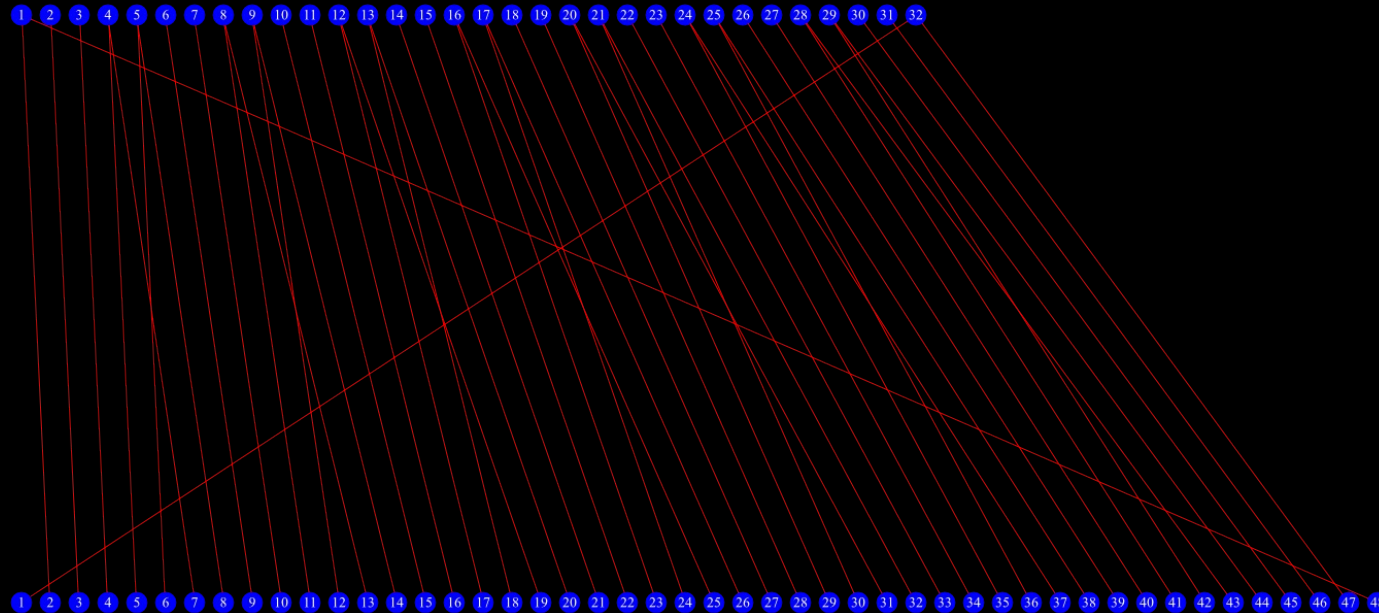


E

Expande la mitad derecha repitiendo algunos bits.

Input: Right (32 bits)

Output: permutedBlock (64 bits)



Cifrado en bloque

DES

Encrypt Block

Round 1

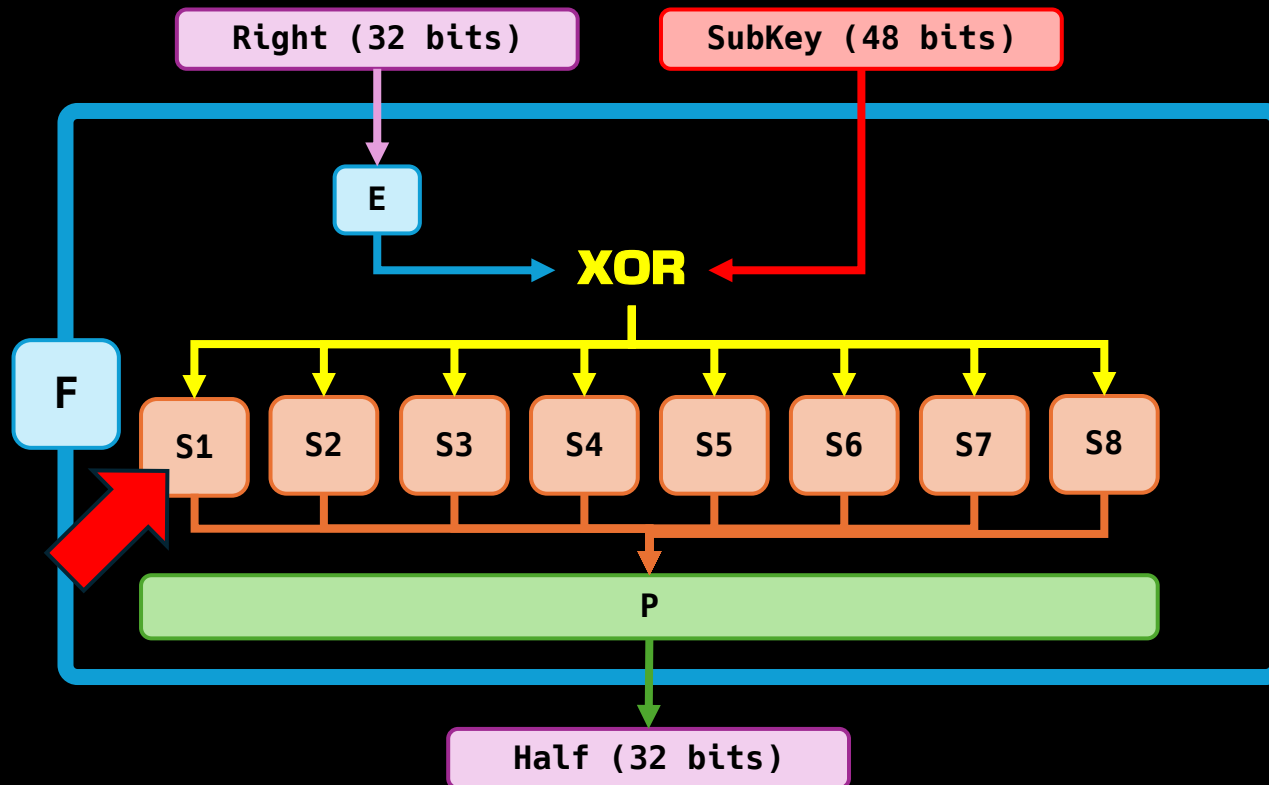
Round 2

...

Round 16



F



Cifrado en bloque

DES

Encrypt Block

Round 1

Round 2

...

Round 16



F



Sx

Cada S-Box transforma 6 bits en 4 bits.

Input: Block (48 bits) **Output:** Block (32 bits)

```
SBoxes(input[48 bits]):  
  for i from 1 to 8 do  
    // 1. Tomar el bloque de 6 bits correspondiente.  
    block := input bits from position ((i - 1) * 6 + 1) to (i * 6)  
  
    // 2. Calcular la fila usando el primer y último bit.  
    rowBits := block[1] || block[6]  
    row := binaryToDecimal(rowBits)  
  
    // 3. Calcular la columna usando los 4 bits centrales.  
    columnBits := block[2] || block[3] || block[4] || block[5]  
    column := binaryToDecimal(columnBits)  
  
    // 4. Aplicar la S-Box correspondiente.  
    value := S[i][row][column]  
  
    // 5. Convertir el valor obtenido a 4 bits.  
    outputBlock[i] := decimalToBinary(value, 4 bits)  
  endfor  
  
  return outputBlock[1] || outputBlock[2] || outputBlock[3] || outputBlock[4] ||  
    outputBlock[5] || outputBlock[6] || outputBlock[7] || outputBlock[8]
```



Cifrado en bloque

DES

Encrypt Block

Round 1

Round 2

...

Round 16



F



Sx

```
SBoxes(input[48 bits]):
  for i from 1 to 8 do
    // 1. Tomar el bloque de 6 bits correspondiente.
    block := input bits from position ((i - 1) * 6 + 1) to (i * 6)

    // 2. Calcular la fila usando el primer y último bit.
    rowBits := block[1] || block[6]
    row := binaryToDecimal(rowBits)

    // 3. Calcular la columna usando los 4 bits centrales.
    columnBits := block[2] || block[3] || block[4] || block[5]
    column := binaryToDecimal(columnBits)

    // 4. Aplicar la S-Box correspondiente.
    value := S[i][row][column]

    // 5. Convertir el valor obtenido a 4 bits.
    outputBlock[i] := decimalToBinary(value, 4 bits)
  endfor

  return outputBlock[1] || outputBlock[2] || outputBlock[3] || outputBlock[4] ||
    outputBlock[5] || outputBlock[6] || outputBlock[7] || outputBlock[8]
```



Cifrado en bloque

DES

Encrypt Block



```
// 4. Aplicar la S-Box correspondiente.  
value := S[i][row][column]
```

S1

S ₁	x0000x	x0001x	x0010x	x0011x	x0100x	x0101x	x0110x	x0111x	x1000x	x1001x	x1010x	x1011x	x1100x	x1101x	x1110x	x1111x
0yyyy0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
0yyyy1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
1yyyy0	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
1yyyy1	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

S5

S ₅	x0000x	x0001x	x0010x	x0011x	x0100x	x0101x	x0110x	x0111x	x1000x	x1001x	x1010x	x1011x	x1100x	x1101x	x1110x	x1111x
0yyyy0	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
0yyyy1	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
1yyyy0	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
1yyyy1	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	13

S2

S ₂	x0000x	x0001x	x0010x	x0011x	x0100x	x0101x	x0110x	x0111x	x1000x	x1001x	x1010x	x1011x	x1100x	x1101x	x1110x	x1111x
0yyyy0	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
0yyyy1	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
1yyyy0	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
1yyyy1	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9

S6

S ₆	x0000x	x0001x	x0010x	x0011x	x0100x	x0101x	x0110x	x0111x	x1000x	x1001x	x1010x	x1011x	x1100x	x1101x	x1110x	x1111x
0yyyy0	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
0yyyy1	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
1yyyy0	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
1yyyy1	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13

S3

S ₃	x0000x	x0001x	x0010x	x0011x	x0100x	x0101x	x0110x	x0111x	x1000x	x1001x	x1010x	x1011x	x1100x	x1101x	x1110x	x1111x
0yyyy0	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
0yyyy1	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
1yyyy0	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
1yyyy1	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12

S7

S ₇	x0000x	x0001x	x0010x	x0011x	x0100x	x0101x	x0110x	x0111x	x1000x	x1001x	x1010x	x1011x	x1100x	x1101x	x1110x	x1111x
0yyyy0	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
0yyyy1	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
1yyyy0	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
1yyyy1	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12

S4

S ₄	x0000x	x0001x	x0010x	x0011x	x0100x	x0101x	x0110x	x0111x	x1000x	x1001x	x1010x	x1011x	x1100x	x1101x	x1110x	x1111x
0yyyy0	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
0yyyy1	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
1yyyy0	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
1yyyy1	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14

S8

S ₈	x0000x	x0001x	x0010x	x0011x	x0100x	x0101x	x0110x	x0111x	x1000x	x1001x	x1010x	x1011x	x1100x	x1101x	x1110x	x1111x
0yyyy0	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
0yyyy1	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
1yyyy0	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
1yyyy1	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11



Cifrado en bloque

DES

Encrypt Block

Round 1

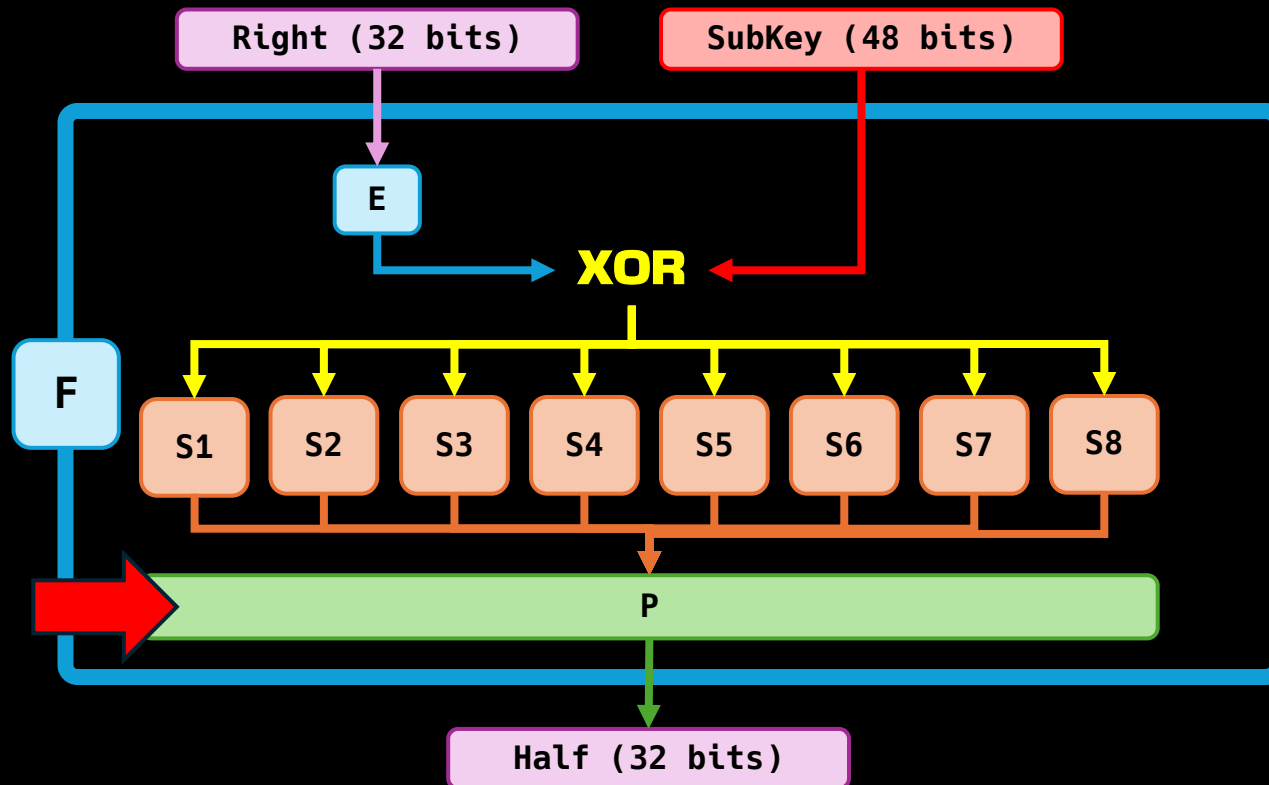
Round 2

...

Round 16



F



Cifrado en bloque

DES

Encrypt Block

Round 1

Round 2

...

Round 16



F



P

Realiza una permutación

Input: Block (32 bits)

Output: permutedBlock (32 bits)

```
P(block[32 bits]):  
  for i from 1 to 32  
    permutedBlock[i] := block[P[i]]  
  endfor  
  return permutedBlock
```

P							
16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25



Cifrado en bloque

DES

Encrypt Block

Round 1

Round 2

...

Round 16



F

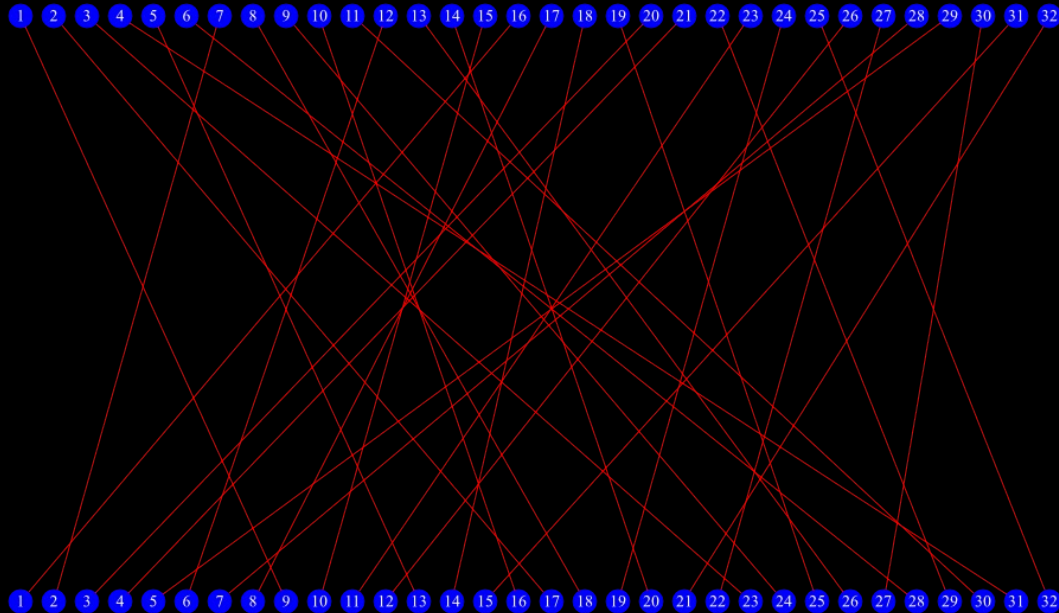


P

Realiza una permutación

Input: Block (32 bits)

Output: permutedBlock (32 bits)



Cifrado en bloque

DES

Encrypt Block

Round 1

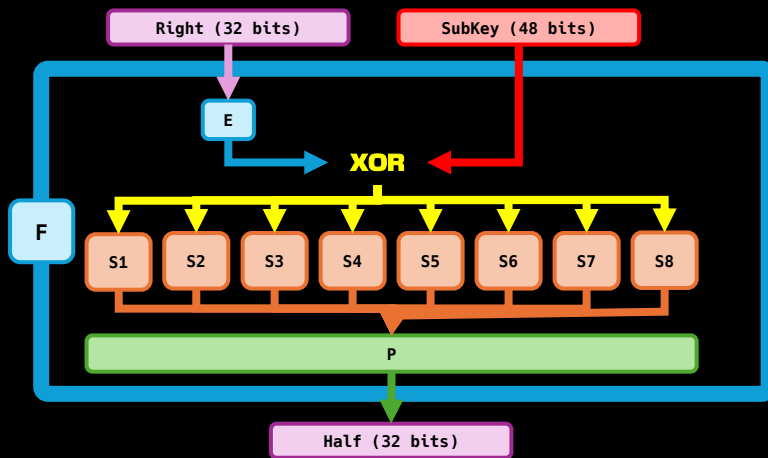
Round 2

...

Round 16



F



```
F(R[32 bits], K[48 bits]):  
  expandedR := E(R)  
  mixed := expandedR xor K  
  substituted := SBoxes(mixed)  
  result := P(substituted)  
  return result
```



Cifrado en bloque

DES

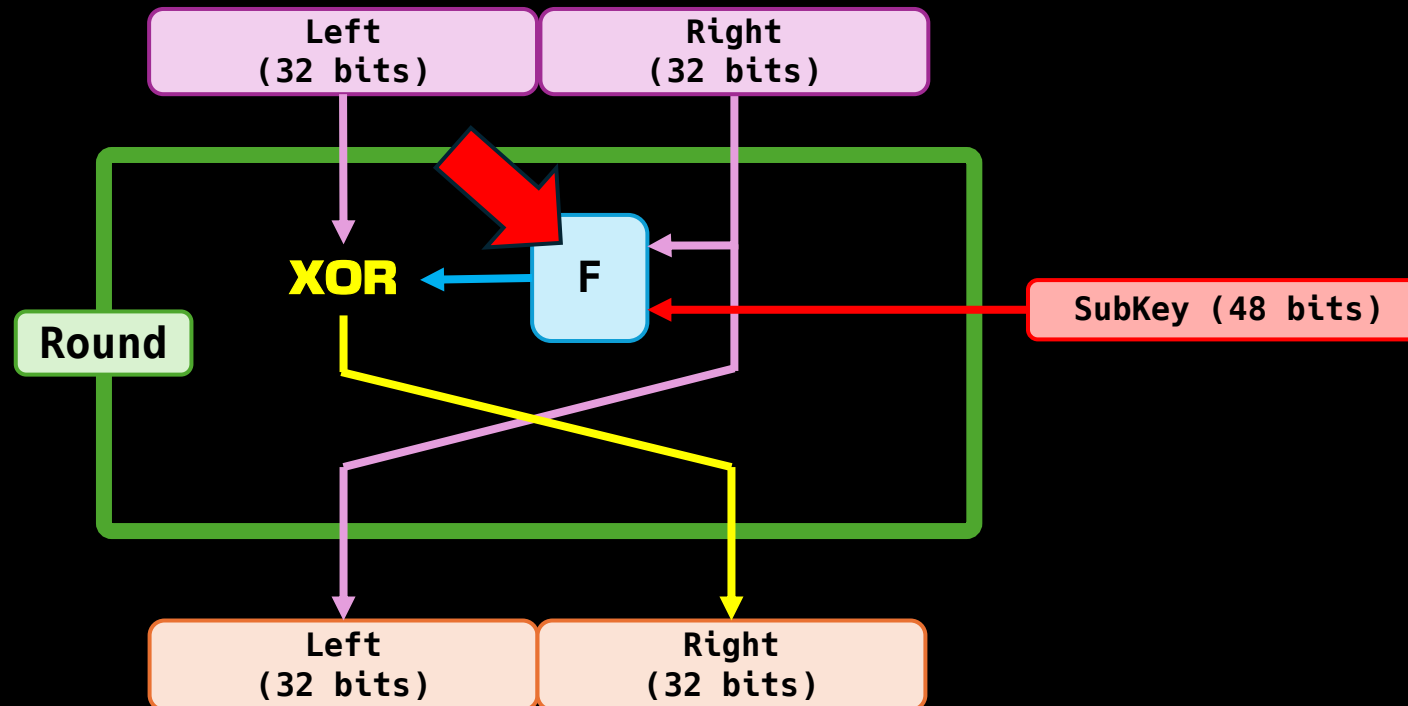
Encrypt Block

Round 1

Round 2

...

Round 16



Cifrado en bloque

DES

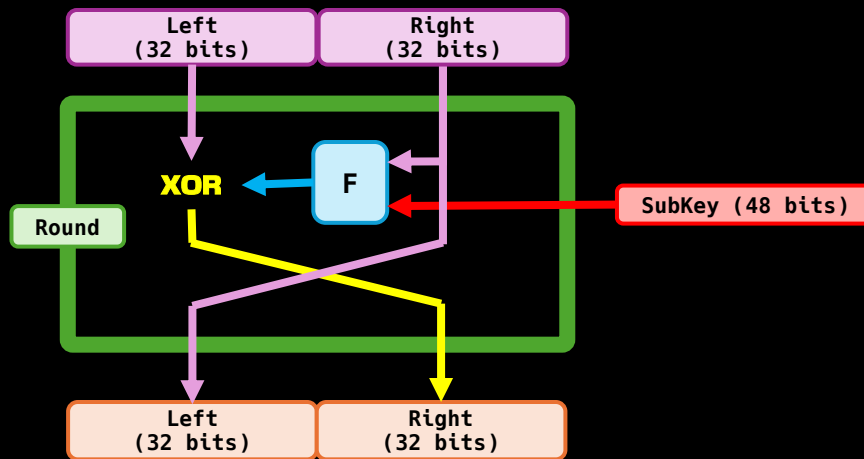
Encrypt Block

Round 1

Round 2

...

Round 16



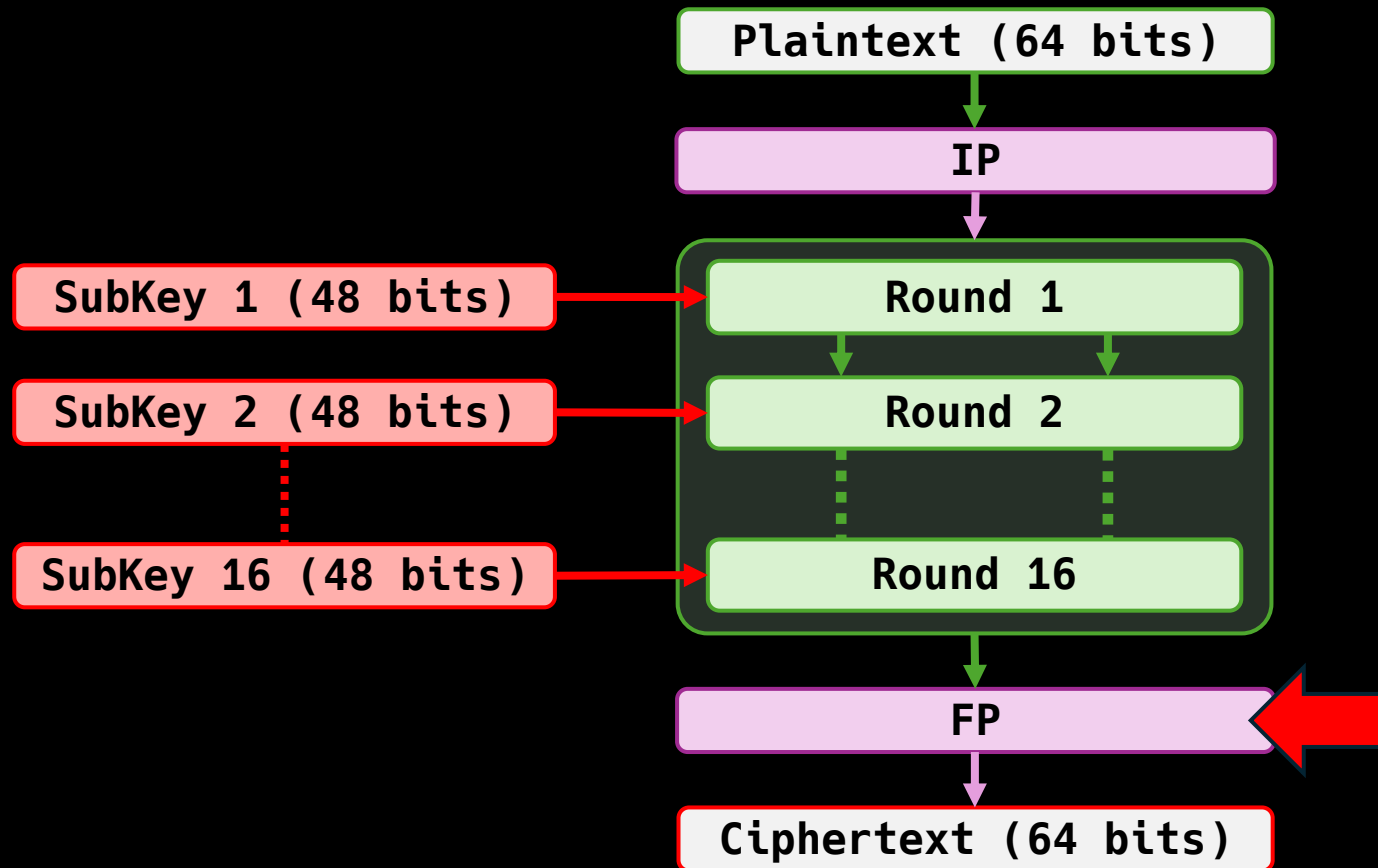
```
FeistelRound(left[32 bits], right[32 bits], K[48 bits]):  
    newLeft := right  
    newRight := left xor F(right, K)  
    return newLeft, newRight
```



Cifrado en bloque

DES

Encrypt Block



Cifrado en bloque

DES

Encrypt Block

FP

FP es la inversa de IP.

Input: Block (64 bits) **Output:** CipherText (64 bits)

```
FP(block[64 bits]):  
  for i from 1 to 64  
    finalBlock[i] := block[FP[i]]  
  endfor  
  return finalBlock
```

IP ⁻¹							
40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25



Cifrado en bloque

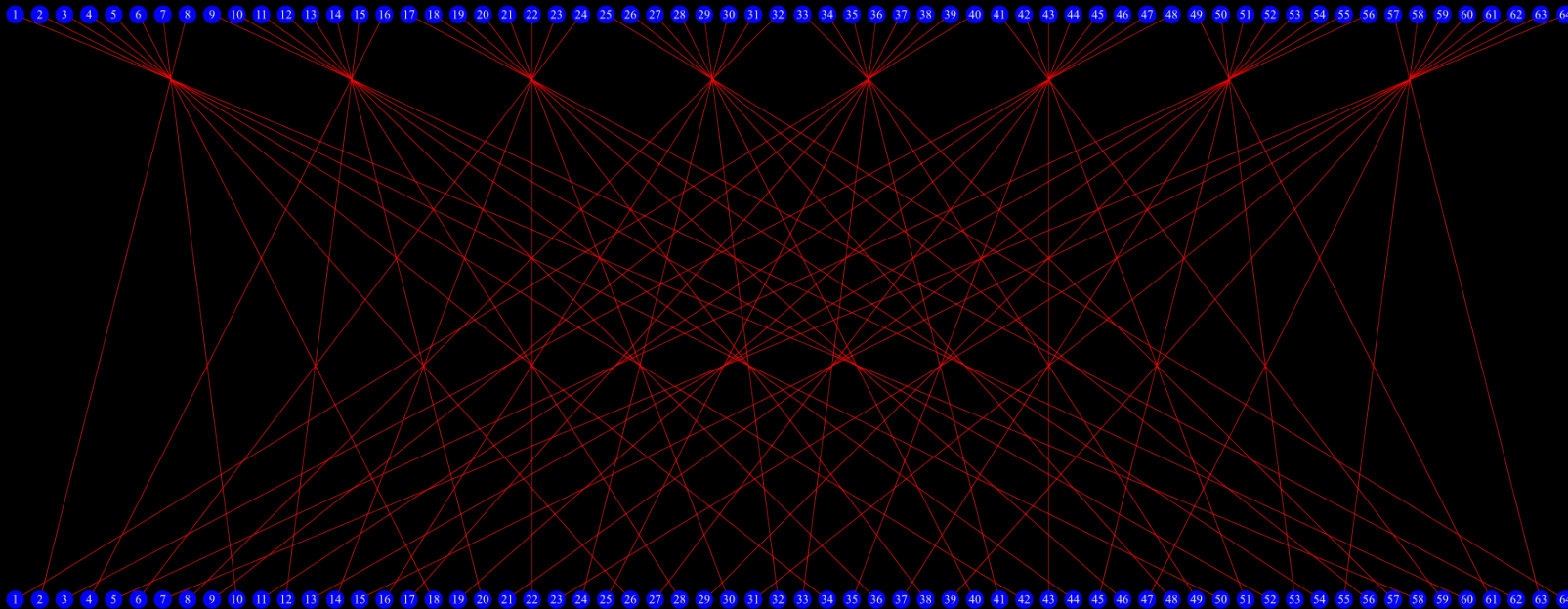
DES

Encrypt Block

FP

FP es la inversa de IP.

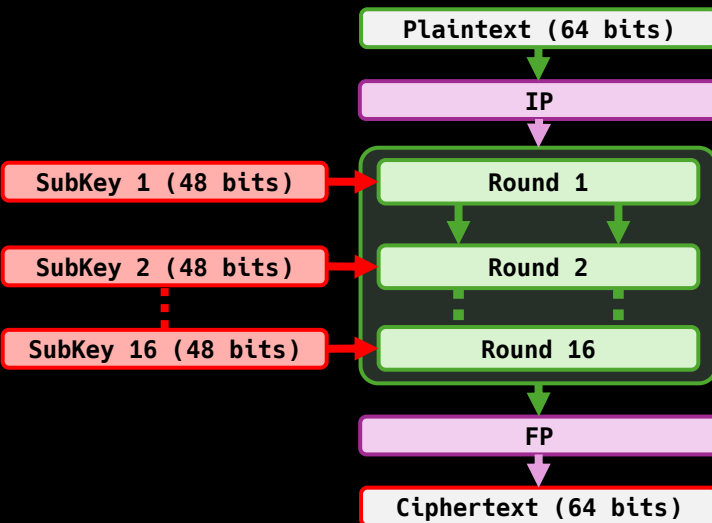
Input: Block (64 bits) **Output:** CipherText (64 bits)



Cifrado en bloque

DES

Encrypt Block

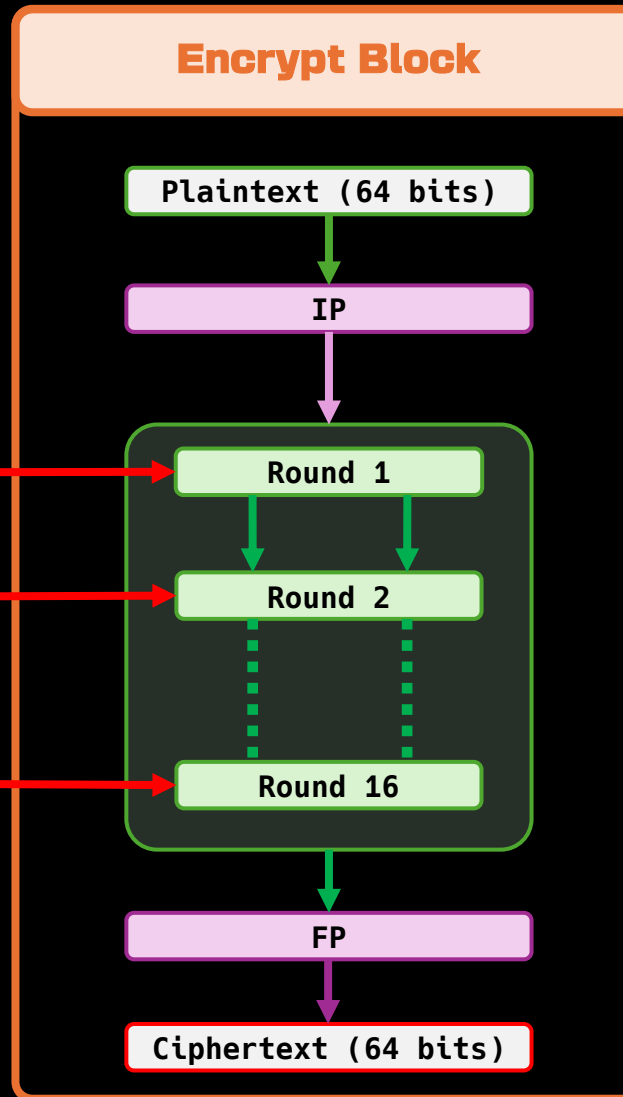
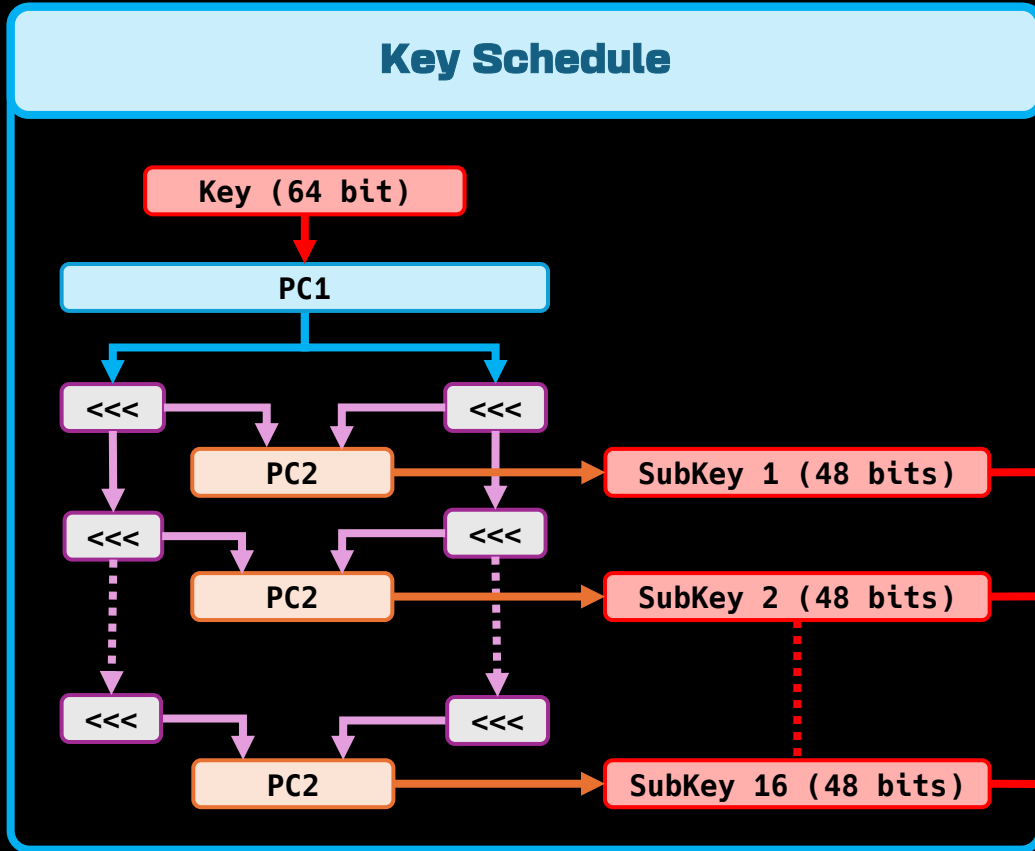


```
KeystreamGenerationAlgorithm(block[64 bits], keys[16]):  
    block := IP(block)  
  
    left := block rightshift 32  
    right := block and 0xFFFFFFFF  
  
    for i from 1 to 16 do  
        left, right := FeistelRound(left, right, keys[i])  
    endfor  
  
    preoutput := (right leftshift 32) or left  
  
    outputBlock := FP(preoutput)  
  
    return outputBlock
```



Cifrado en bloque

DES



Cifrado en bloque

DES



3DES

AES



Cifrado en bloque

3DES

Cifrado 3DES

Publicado en 1998 como evolución de DES.

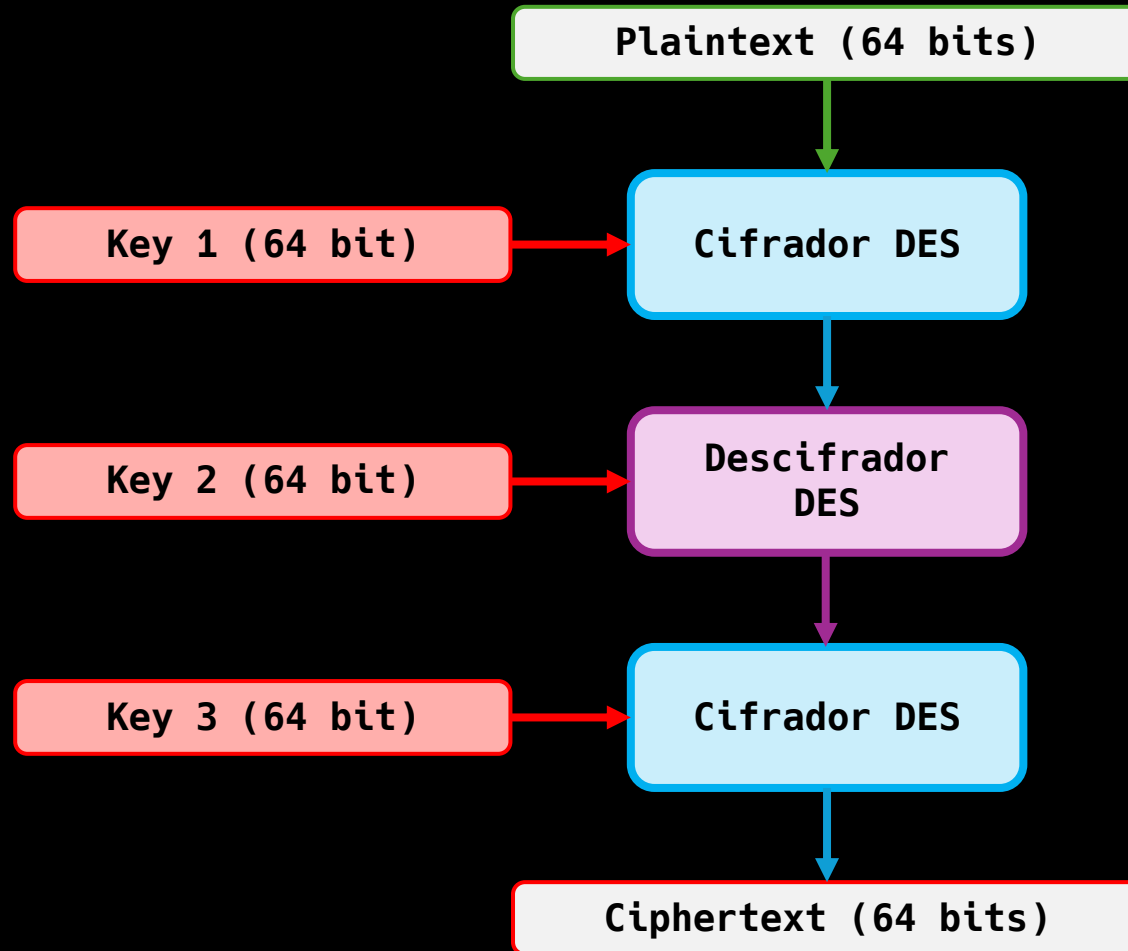
Aplica el cifrado DES tres veces para aumentar la seguridad frente al cifrado original.

Fue muy usado en sistemas bancarios y financieros, pero hoy está obsoleto.



Cifrado en bloque

3DES



Cifrado en bloque

DES



3DES



AES



Cifrado en bloque

AES

Cifrado AES (el moderno)

Publicado en 2001.

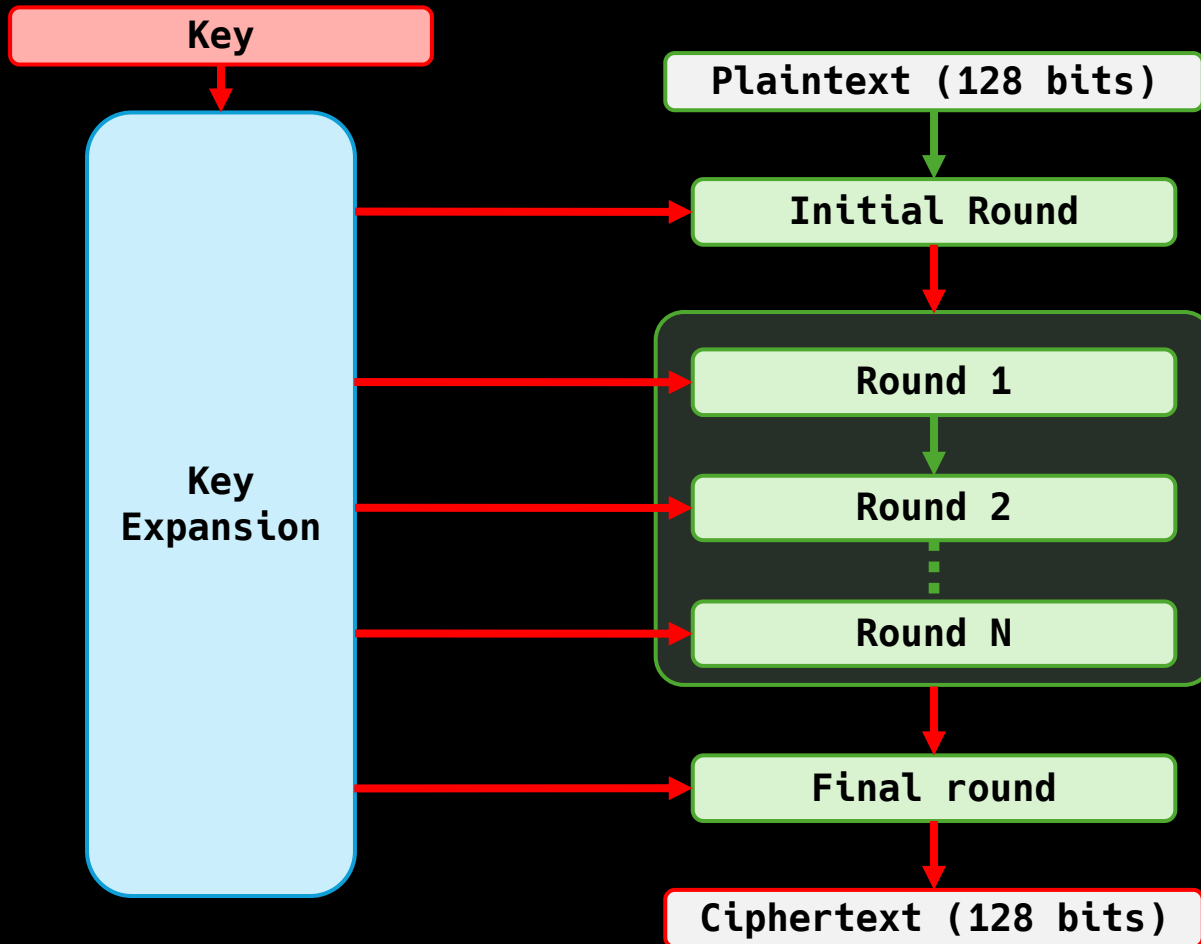
Es el sucesor de DES.

Muy seguro, eficiente y ampliamente usado actualmente en TLS, VPNs, discos cifrados, Wi-Fi y muchos sistemas modernos.



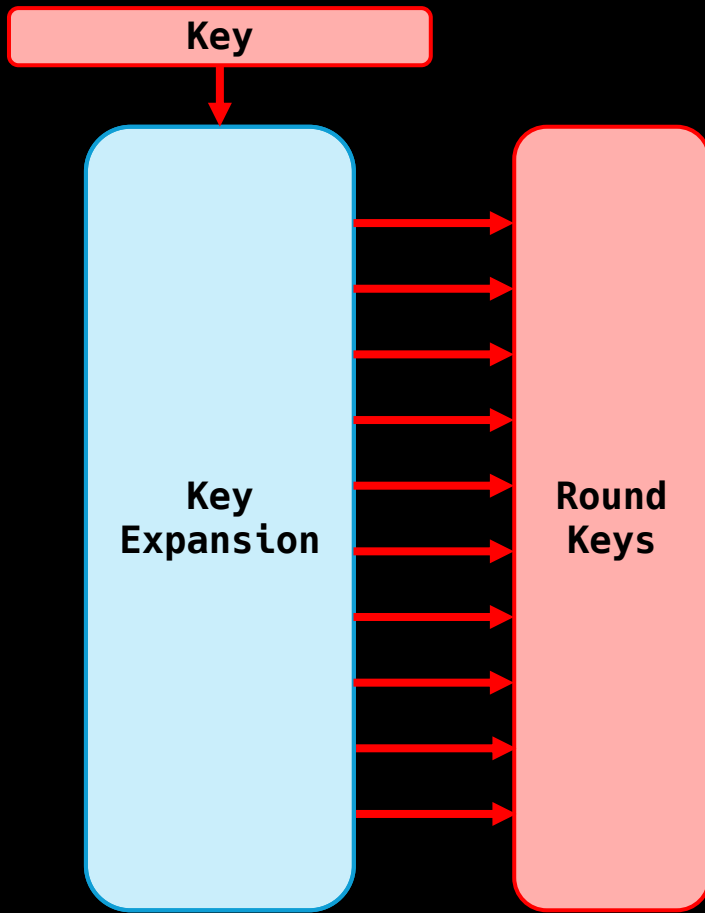
Cifrado en bloque

AES



Cifrado en bloque

AES



La **Key** puede ser de 128, 192 o 256 bits
Y las **Round key** son de 32 bits

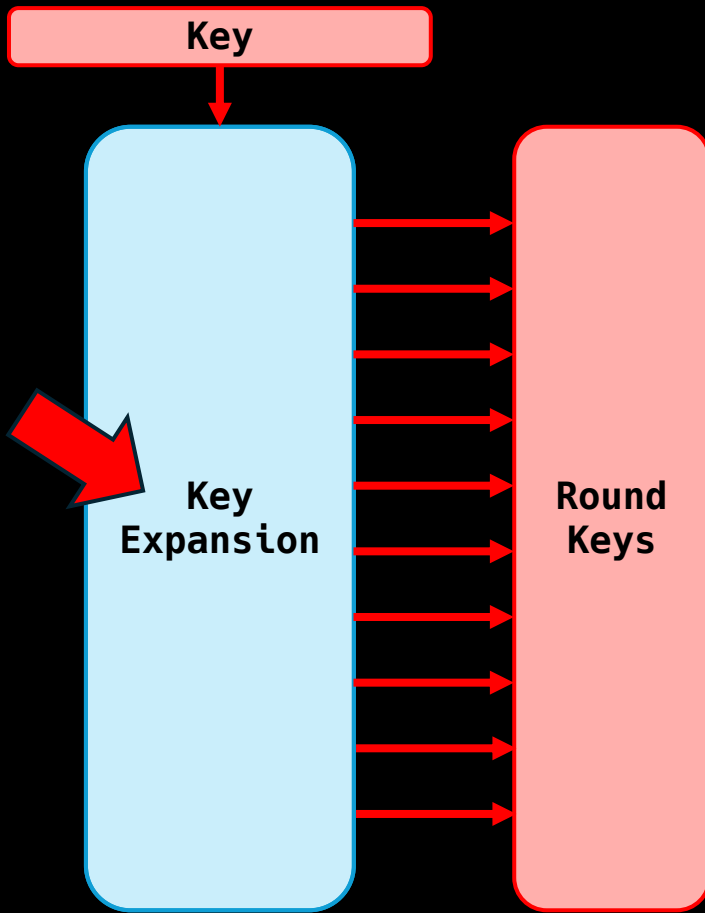
Internamente **Key Expansion** va a dividir
la **Key** en words de 32 bits

Además, **Key Expansion** realiza una serie
de rondas para generar cada **Round key**
de manera individual



Cifrado en bloque

AES

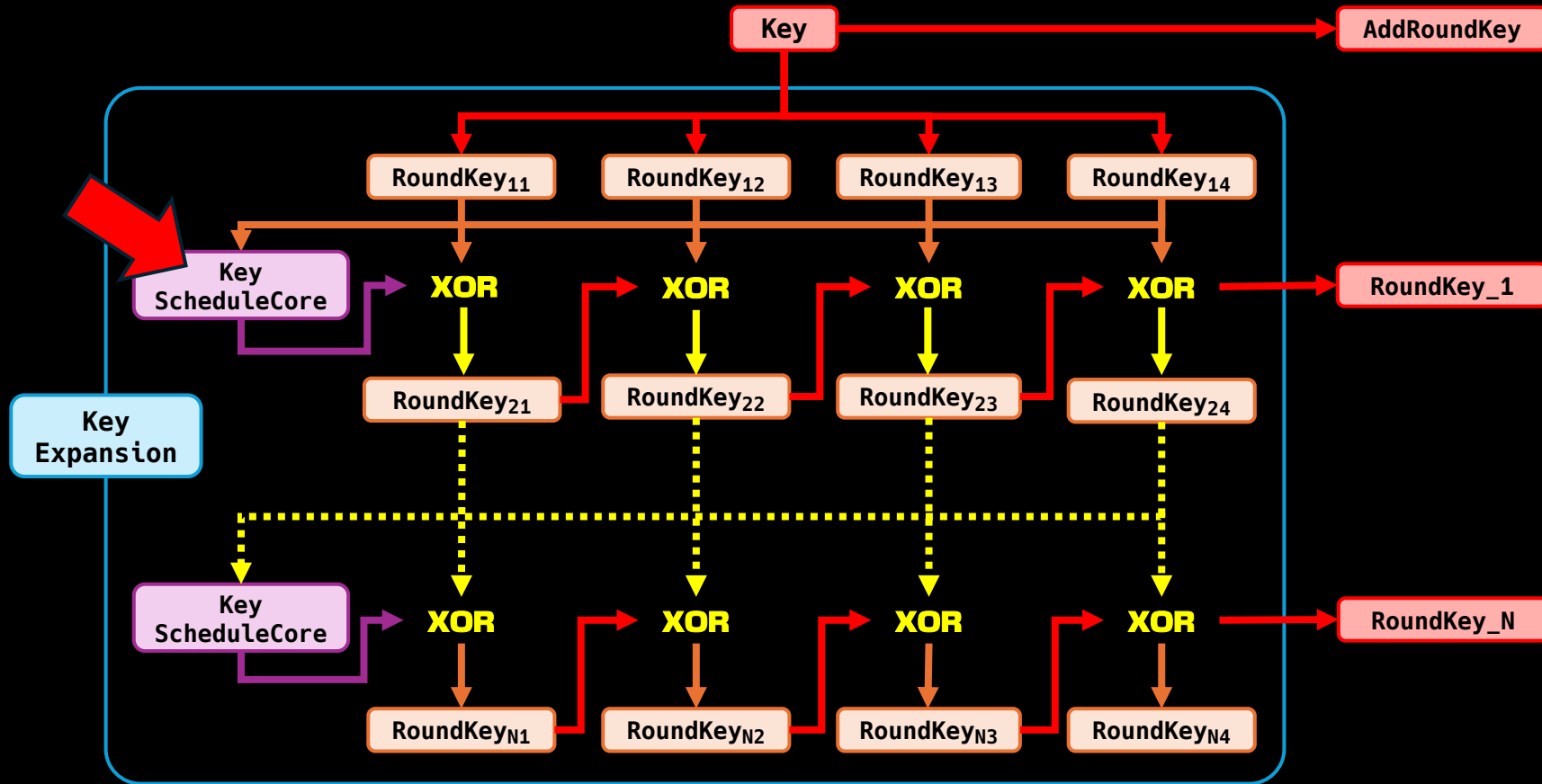


```
AESParameters(key):  
    Nb := 4  
  
    if key length = 128 bits then  
        Nk := 4  
        Nr := 10  
    else if key length = 192 bits then  
        Nk := 6  
        Nr := 12  
    else if key length = 256 bits then  
        Nk := 8  
        Nr := 14  
    endif  
  
    return Nb, Nk, Nr
```



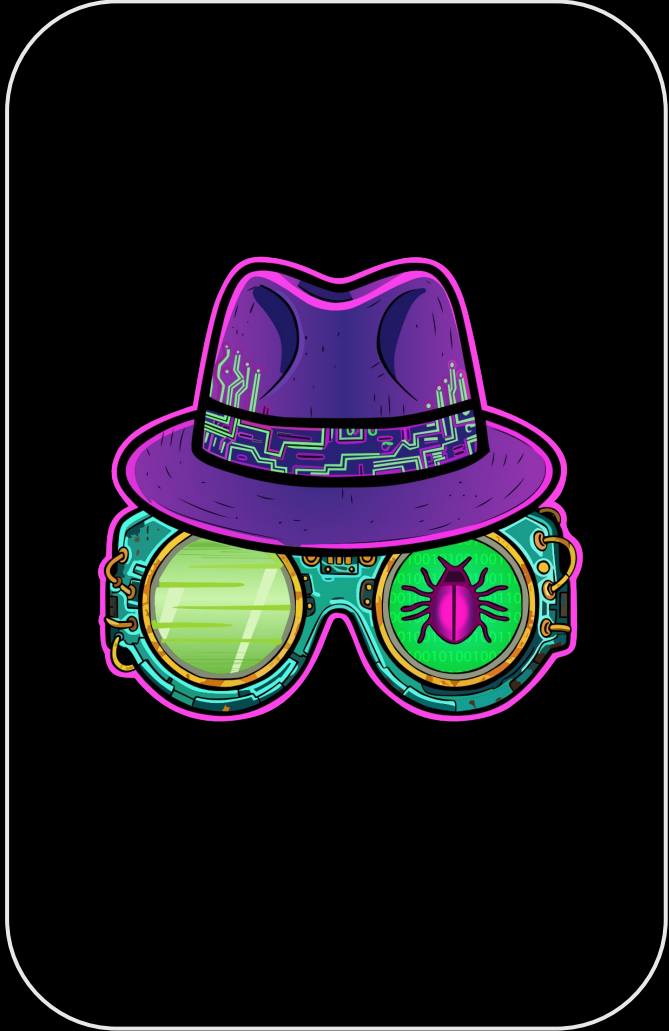
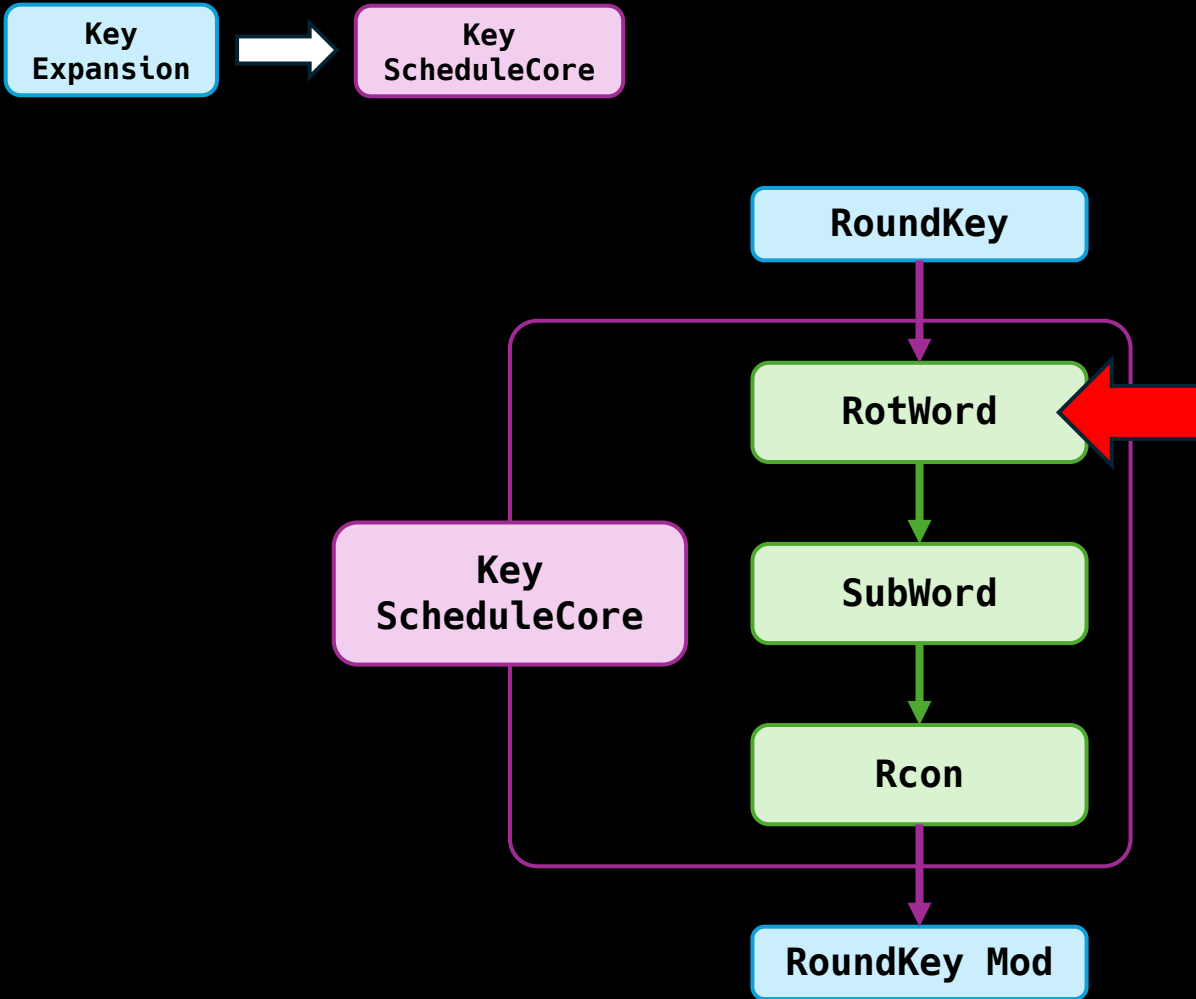
Cifrado en bloque

AES



Cifrado en bloque

AES



Cifrado en bloque

AES



Rota circularmente un word (4 bytes) hacia la izquierda

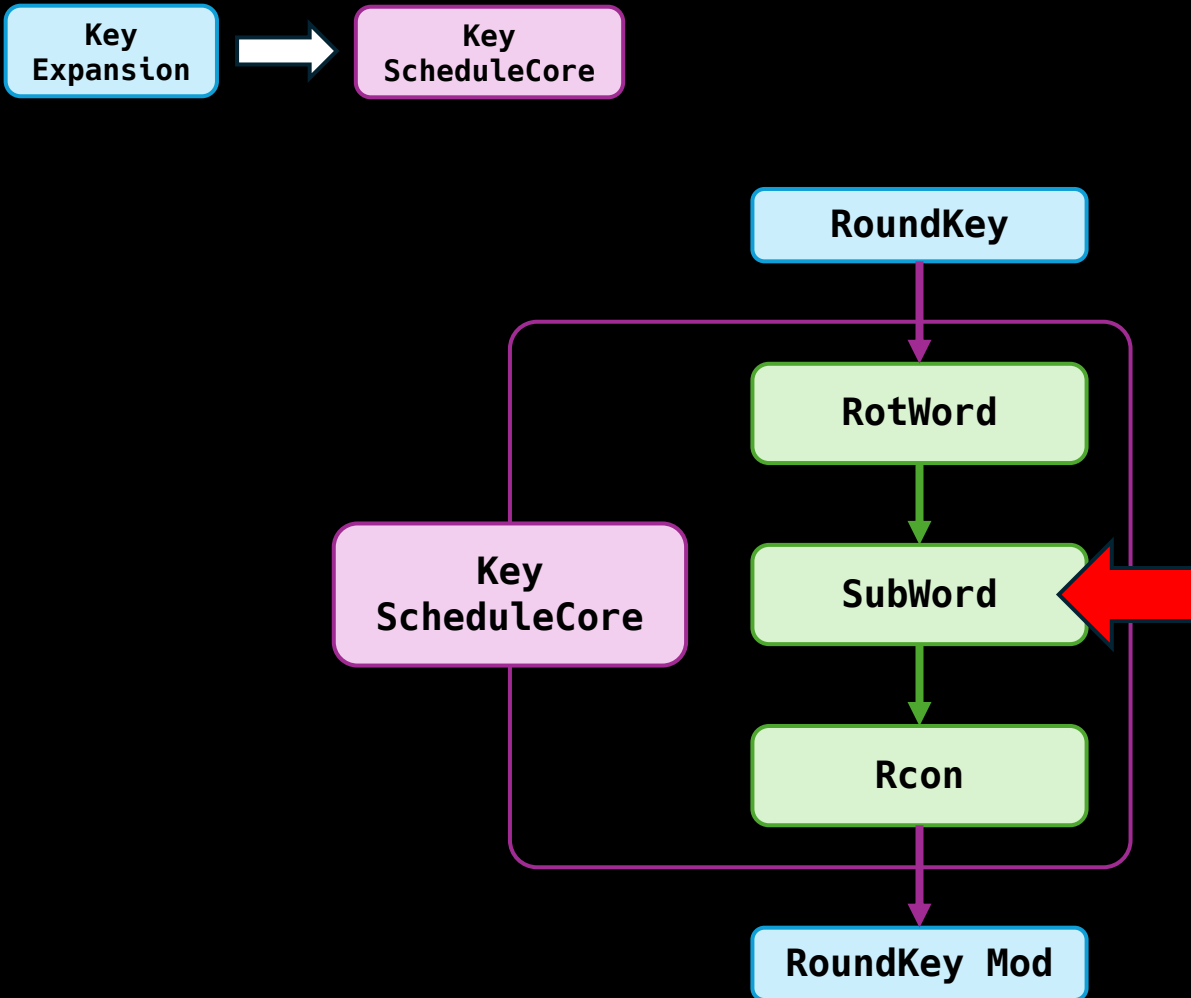
Input: word (4 bytes) **Output:** rotatedWord (4 bytes)

```
RotWord(word[4 bytes]):  
    rotatedWord[0] := word[1]  
    rotatedWord[1] := word[2]  
    rotatedWord[2] := word[3]  
    rotatedWord[3] := word[0]  
  
    return rotatedWord
```



Cifrado en bloque

AES



Cifrado en bloque

AES



Aplica la S-Box de AES a cada byte de una palabra.

Input: word (4 bytes)

Output: substitutedWord (4 bytes)

```
SubWord(word[4 bytes]):  
  for i from 0 to 3 do  
    result[i] := SBox[word[i]]  
  endfor  
  
  return result
```

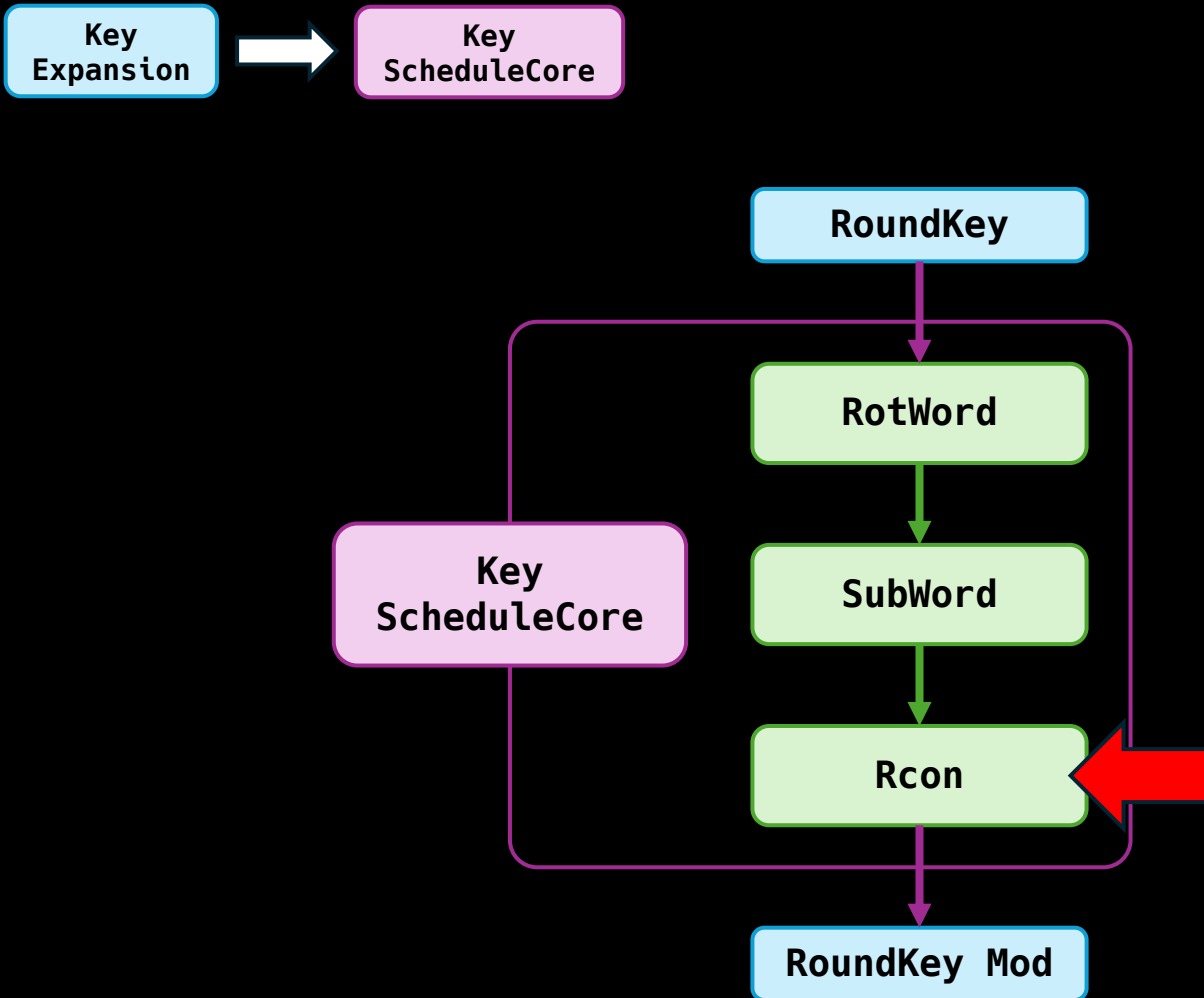
AES S-box																
	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
10	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
20	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
30	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
40	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
50	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
60	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
70	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
80	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
90	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
a0	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
b0	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
c0	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
d0	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
e0	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
f0	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

The column is determined by the least significant nibble, and the row by the most significant nibble. For example, the value $9a_{16}$ is converted into $b8_{16}$.



Cifrado en bloque

AES



Cifrado en bloque

AES



Devuelve la constante de ronda correspondiente

La constante es el primer byte, los otros tres bytes son 0x00

Input: Índice ronda **Output:** word (4 bytes)

```
Rcon(i):  
    return RCON[i]
```



Cifrado en bloque

AES

Key
Expansion

Key
ScheduleCore

RoundKey

RotWord

SubWord

Rcon

RoundKey Mod

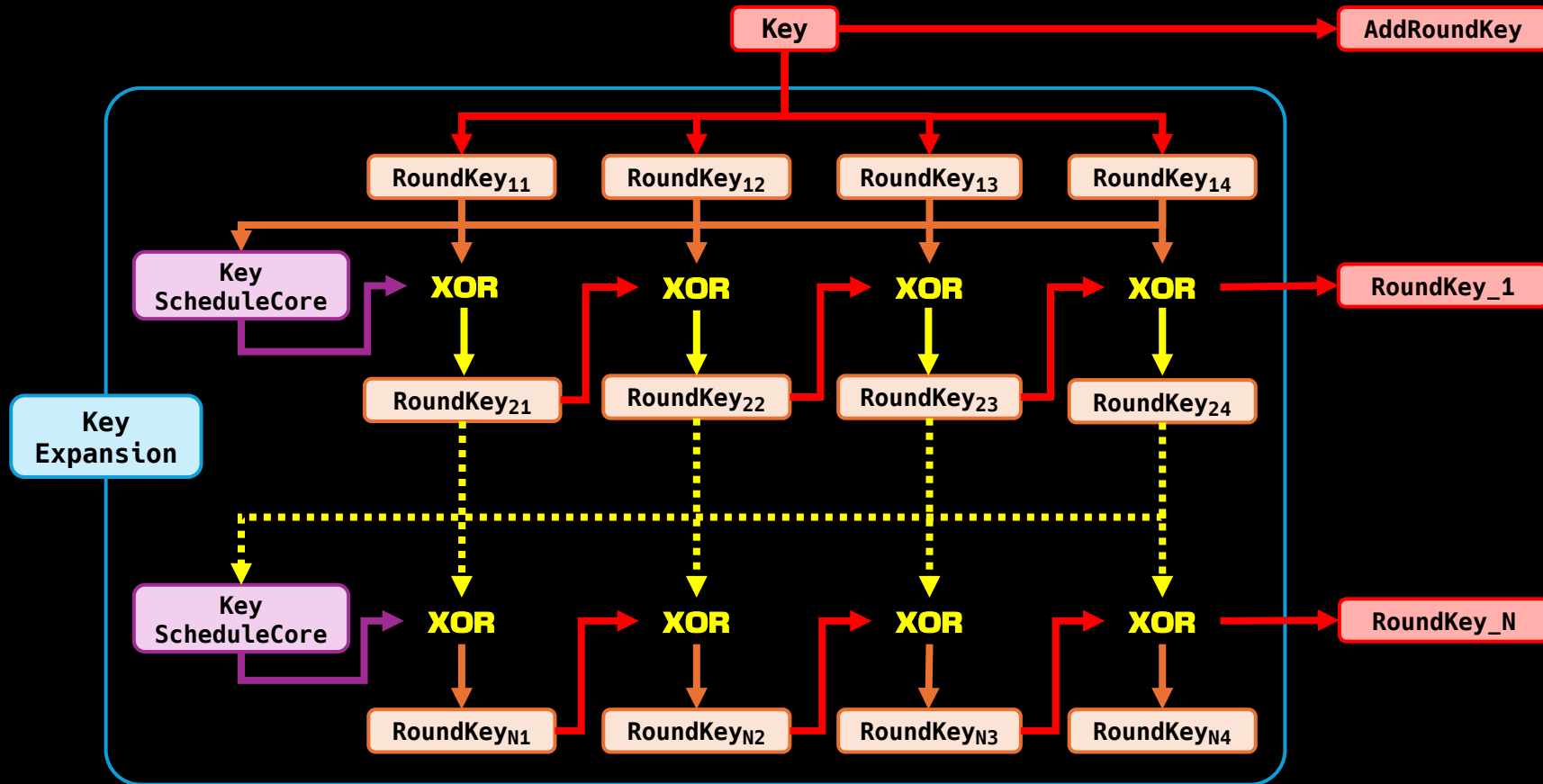
Key
ScheduleCore

```
KeyScheduleCore(word[4 bytes], round):  
    word := RotWord(word)  
    word := SubWord(word)  
    word := word xor Rcon(round)  
    return word
```



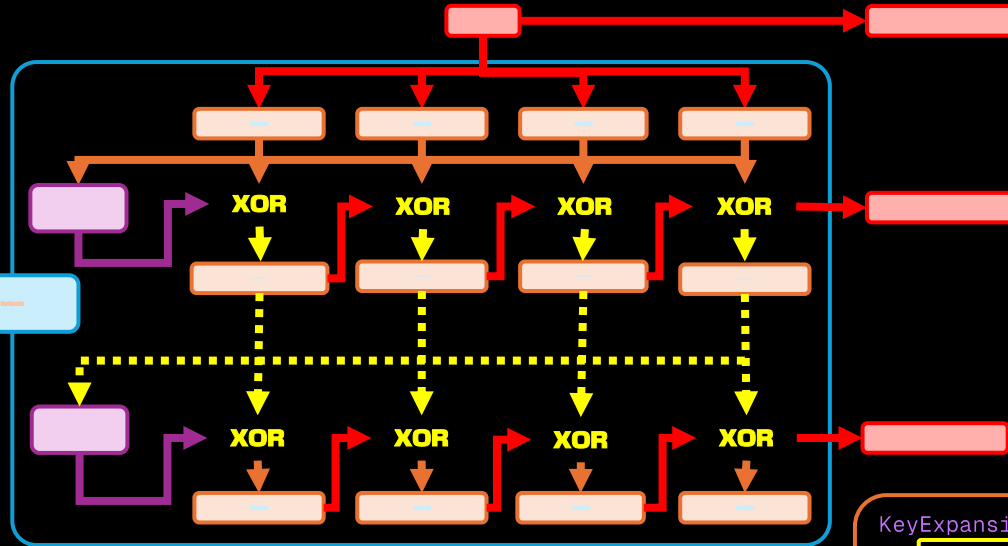
Cifrado en bloque

AES



Cifrado en bloque

AES



Obtener los parámetros de AES según el tamaño de clave.

KeyExpansion(key):

`Nb, Nk, Nr := AESParameters(key)`

```
for i from 0 to Nk - 1 do
    word[i] := key bytes from position (4 * i) to (4 * i + 3)
endfor
```

`totalWords := Nb * (Nr + 1)`

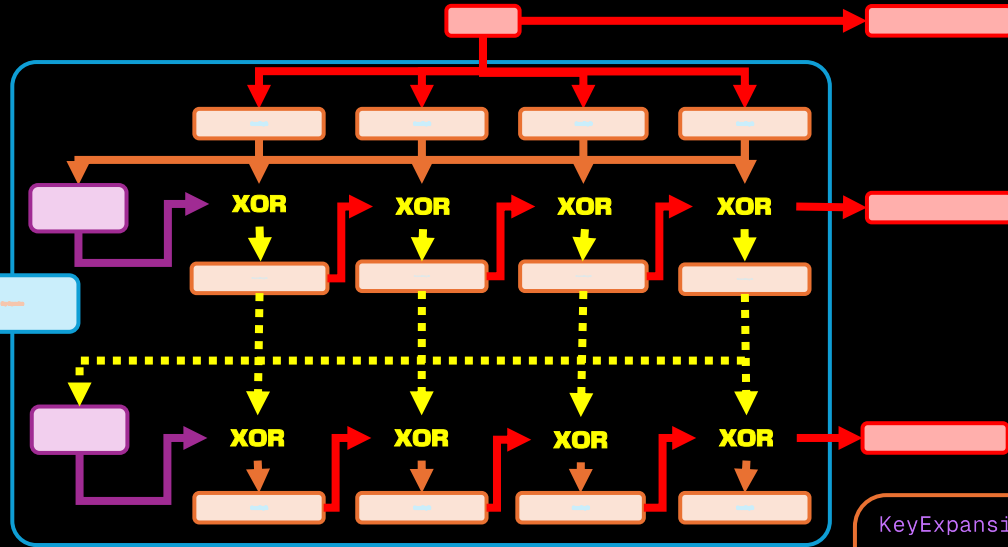
```
for i from Nk to totalWords - 1 do
    temp := word[i - 1]
    if i mod Nk = 0 then
        temp := KeyScheduleCore(temp, i / Nk)
    endif
    word[i] := word[i - Nk] xor temp
endfor
```

`return word, Nr`



Cifrado en bloque

AES



Copiar la clave original en las primeras N_k words.

AES-128 -> 4 words

AES-192 -> 6 words

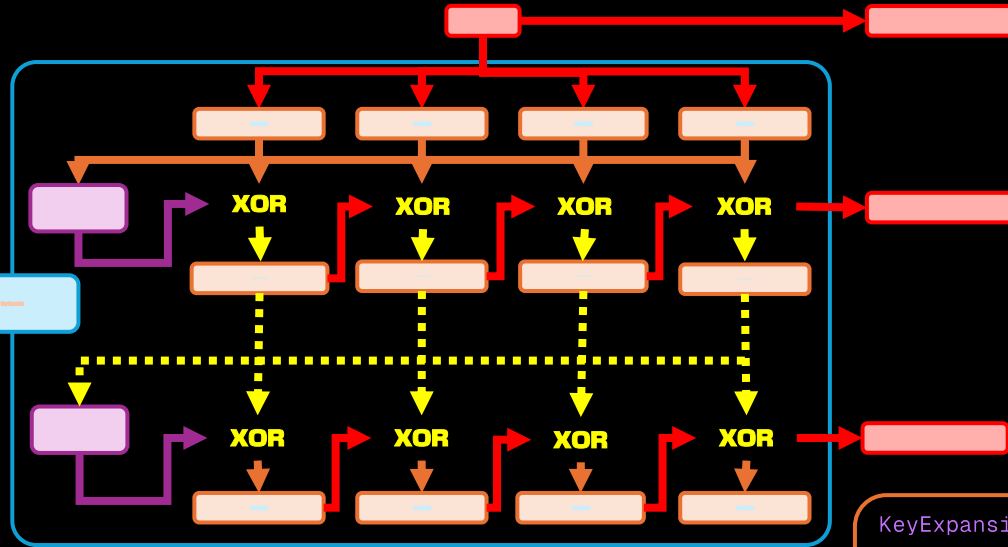
AES-256 -> 8 words

```
KeyExpansion(key):  
    Nb, Nk, Nr := AESParameters(key)  
  
    for i from 0 to Nk - 1 do  
        word[i] := key bytes from position (4 * i) to (4 * i + 3)  
    endfor  
  
    totalWords := Nb * (Nr + 1)  
  
    for i from Nk to totalWords - 1 do  
        temp := word[i - 1]  
        if i mod Nk = 0 then  
            temp := KeyScheduleCore(temp, i / Nk)  
        endif  
        word[i] := word[i - Nk] xor temp  
    endfor  
  
    return word, Nr
```



Cifrado en bloque

AES



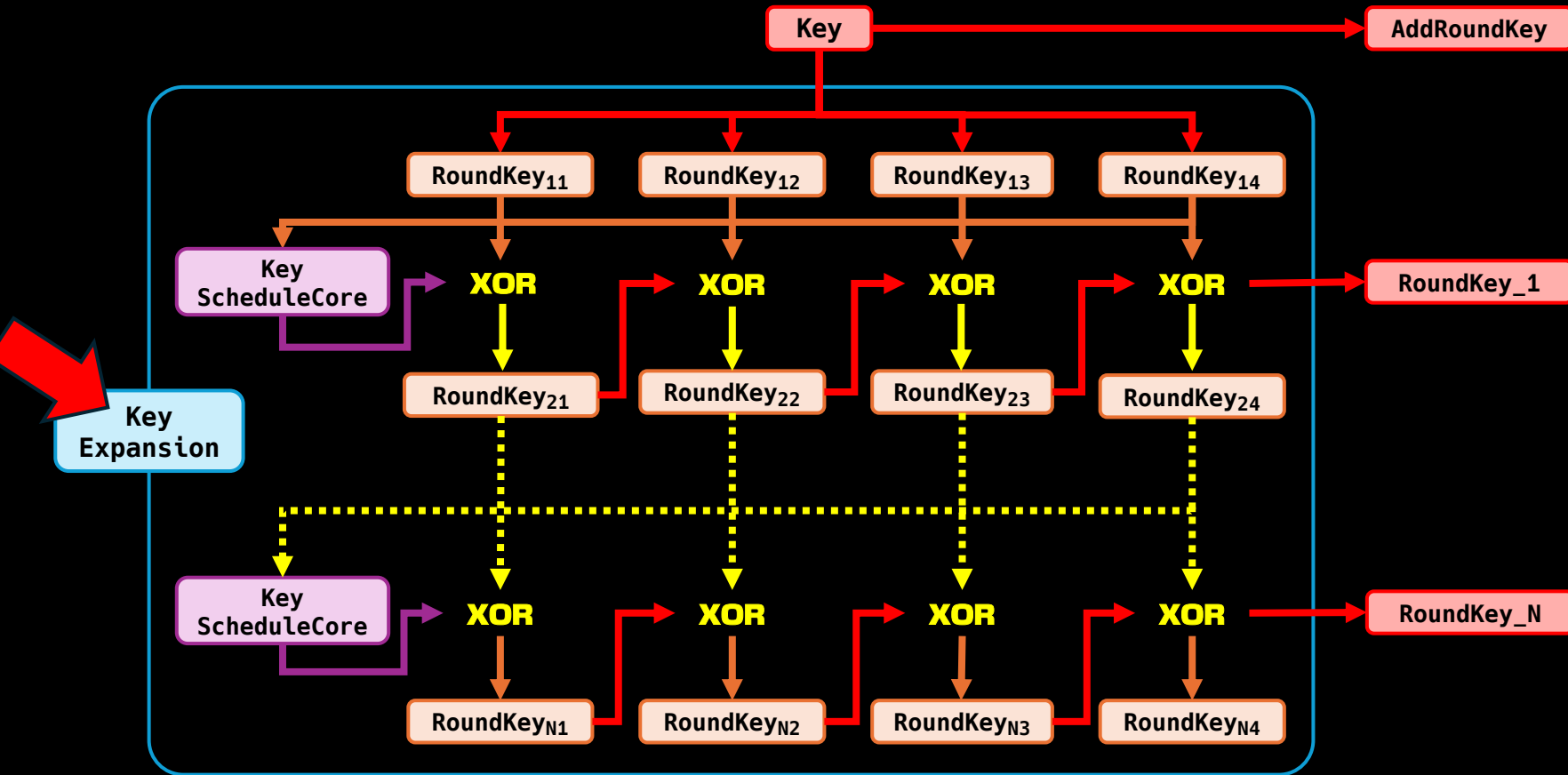
Generar las palabras necesarias para formar todas las round keys.

```
KeyExpansion(key):  
    Nb, Nk, Nr := AESParameters(key)  
  
    for i from 0 to Nk - 1 do  
        word[i] := key bytes from position (4 * i) to (4 * i + 3)  
    endfor  
  
    totalWords := Nb * (Nr + 1)  
  
    for i from Nk to totalWords - 1 do  
        temp := word[i - 1]  
        if i mod Nk = 0 then  
            temp := KeyScheduleCore(temp, i / Nk)  
        endif  
        word[i] := word[i - Nk] xor temp  
    endfor  
  
    return word, Nr
```



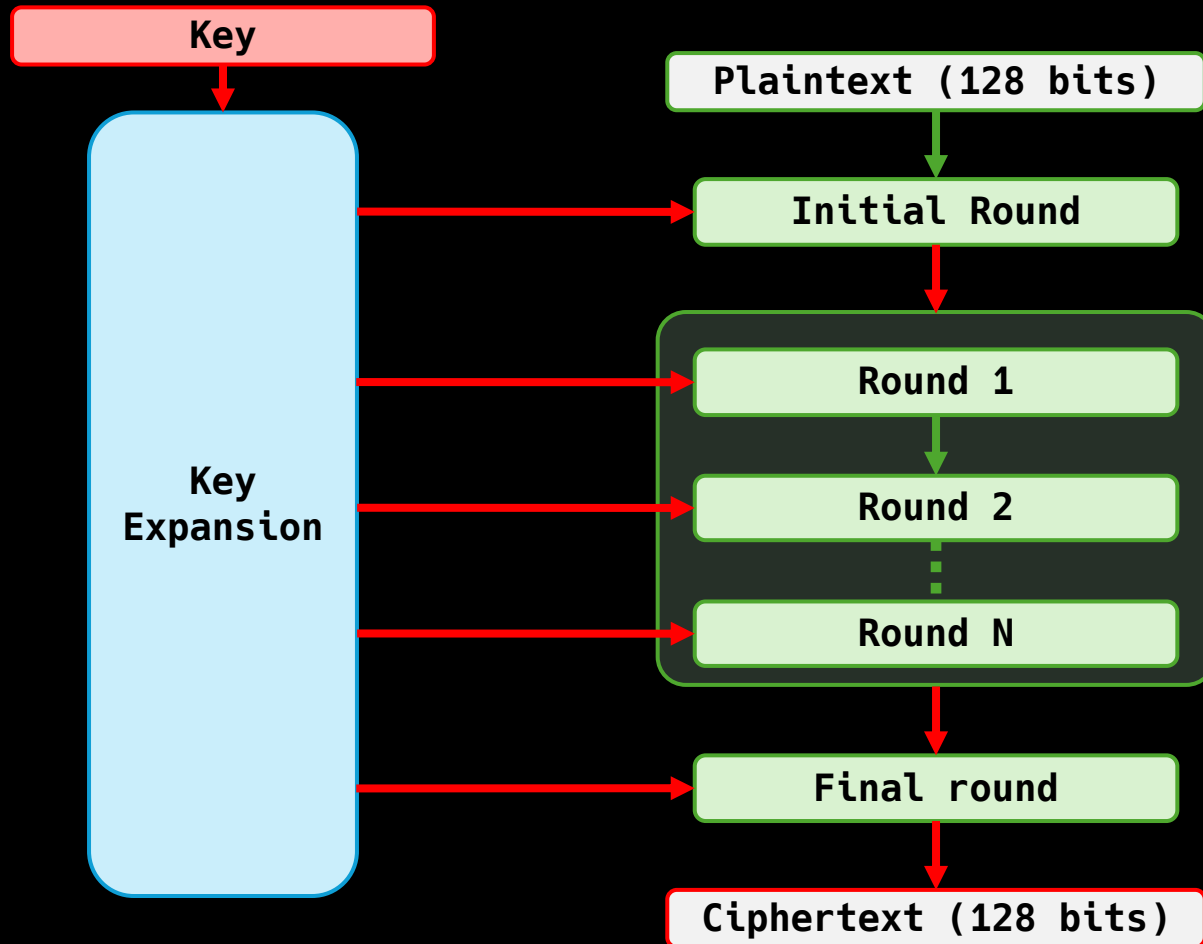
Cifrado en bloque

AES



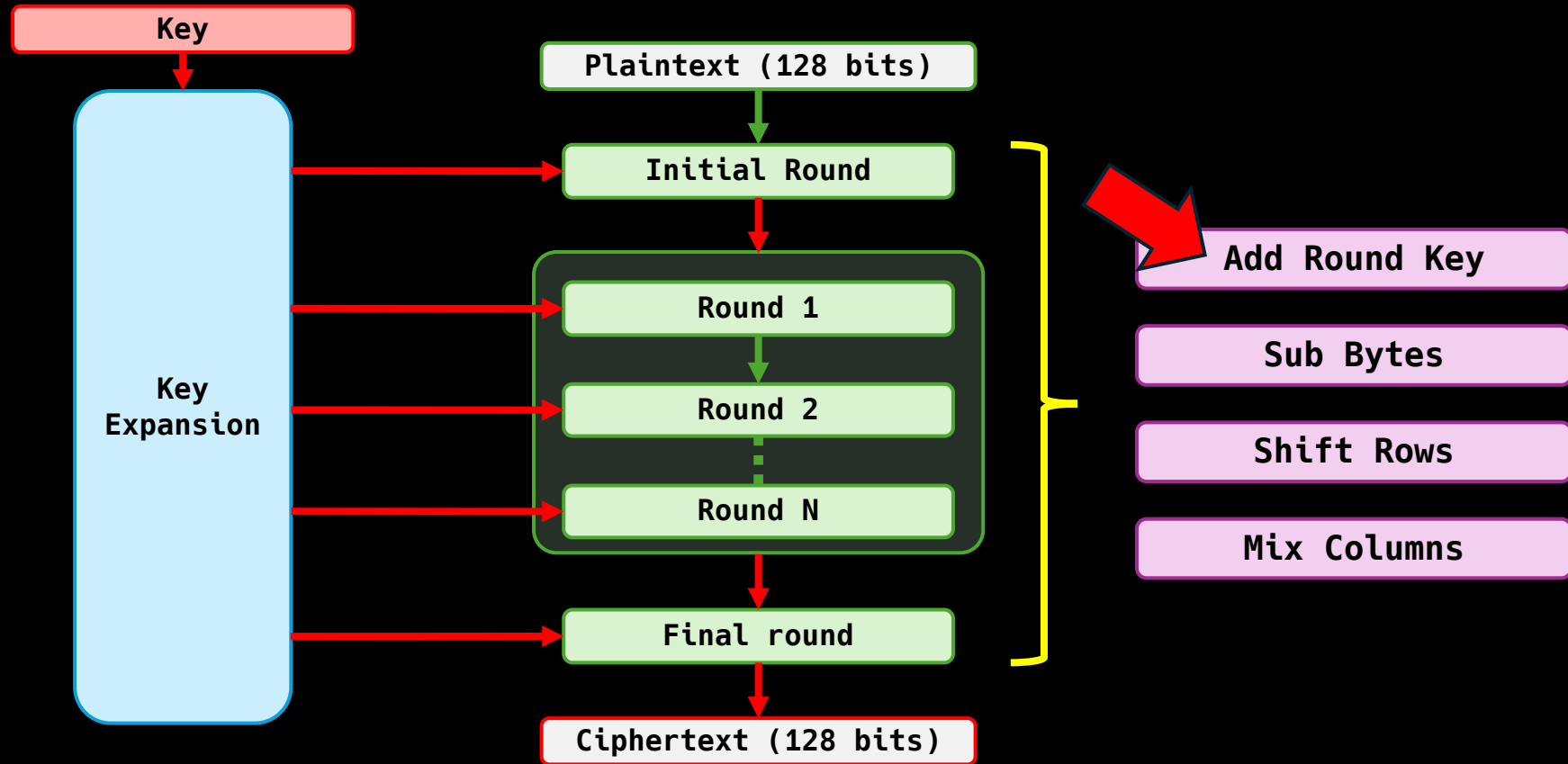
Cifrado en bloque

AES



Cifrado en bloque

AES

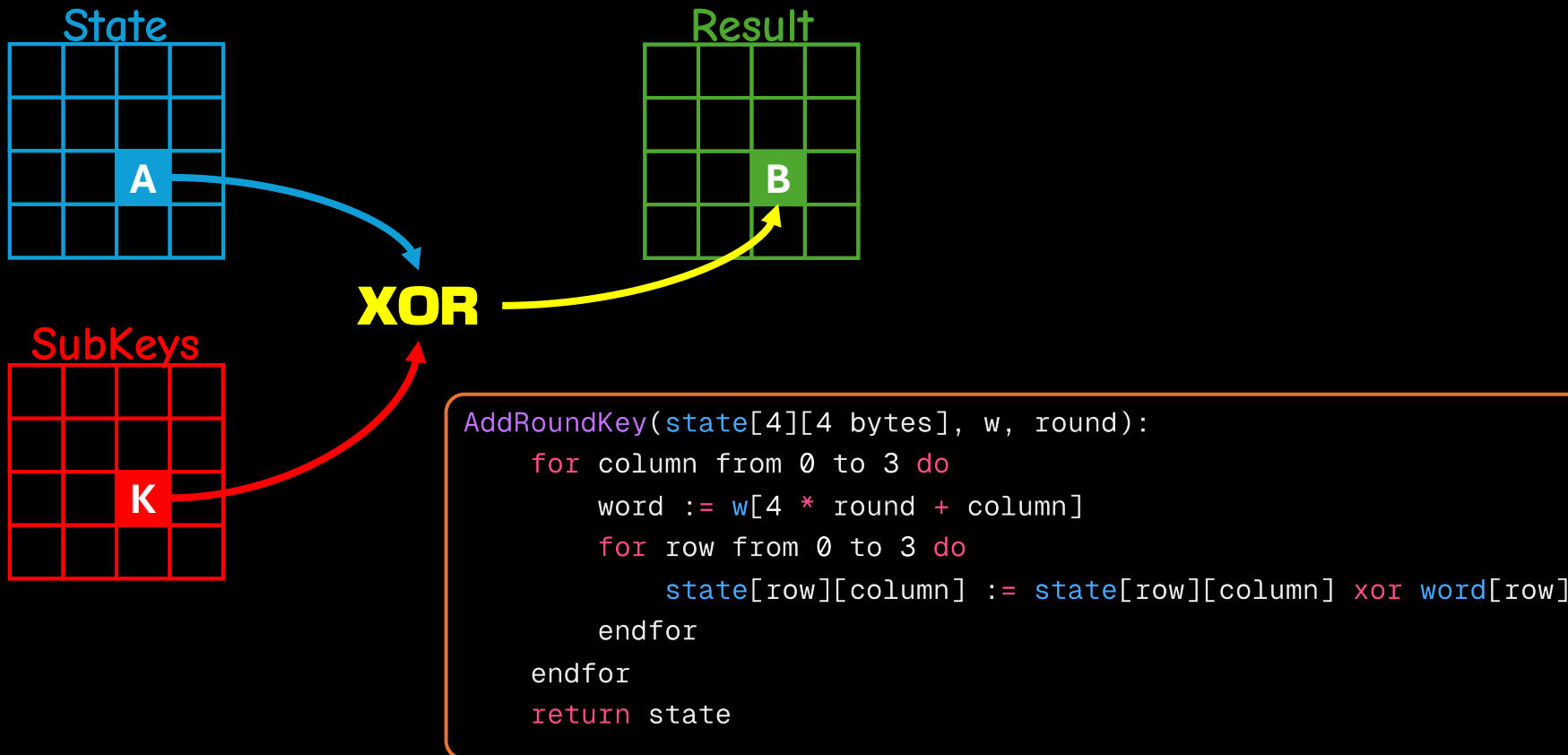


Cifrado en bloque

AES

Add Round Key

Mezcla el State con el round key correspondiente.



AES

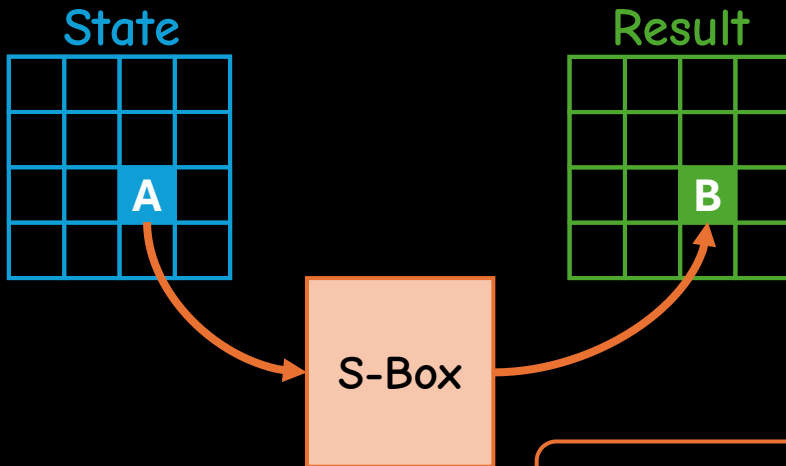


Cifrado en bloque

AES

Sub Bytes

Sustituye cada byte del State usando la **S-Box** de AES

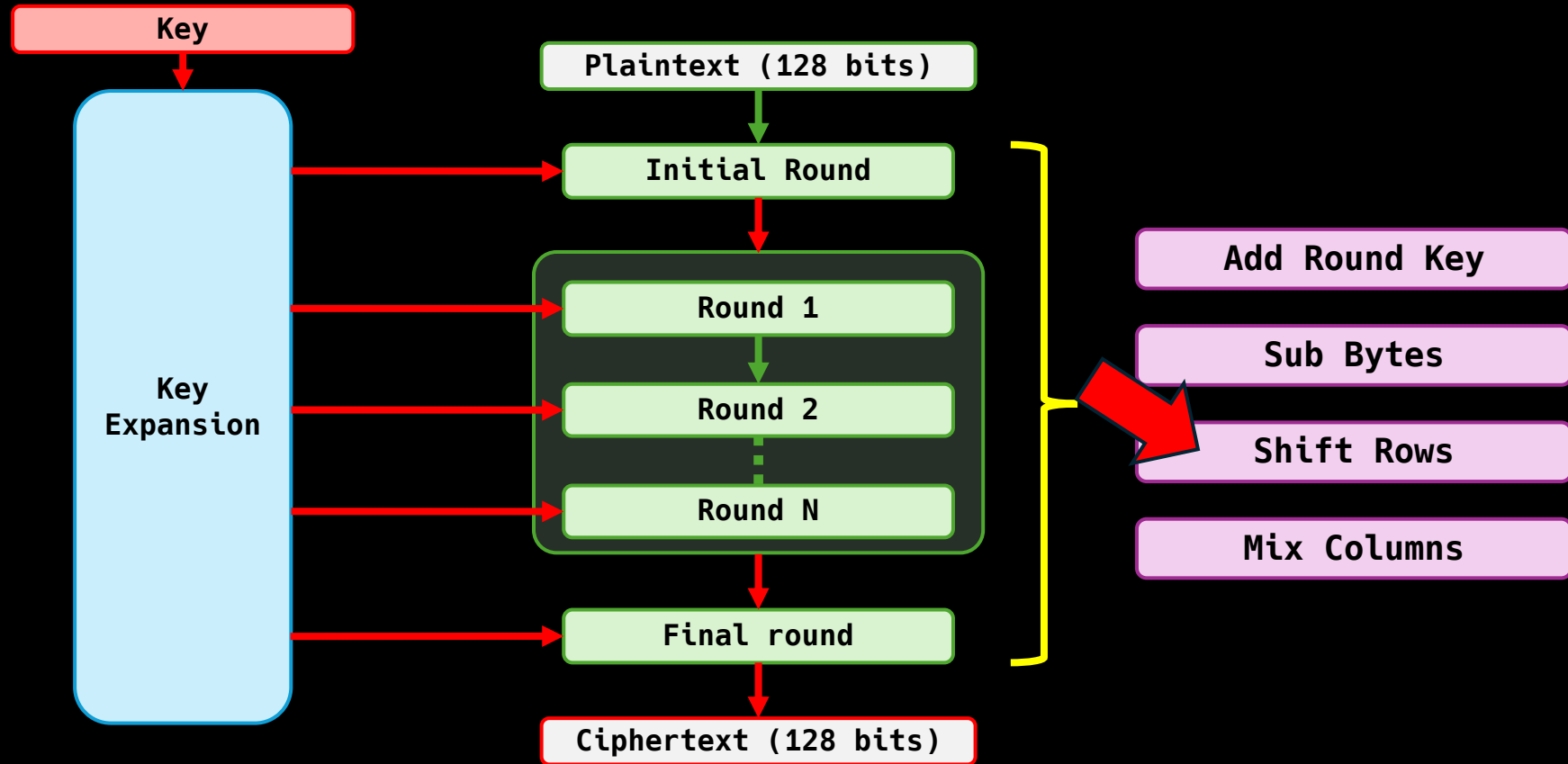


```
SubBytes(state[4][4 bytes]):  
  for row from 0 to 3 do  
    for column from 0 to 3 do  
      state[row][column] := SBox[state[row][column]]  
    endfor  
  endfor  
  return state
```



Cifrado en bloque

AES



Cifrado en bloque

AES

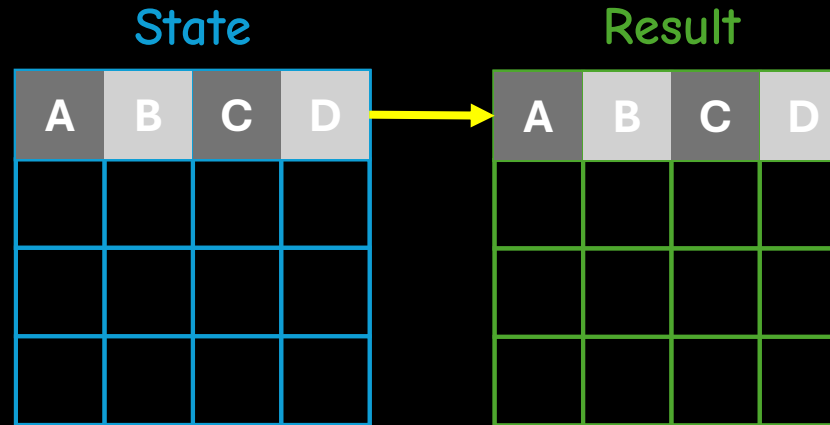
Shift Rows

Desplaza circularmente las filas del State

```
ShiftRows(state[4][4 bytes]):
```

```
state[0] := state[0] leftrotate 0  
state[1] := state[1] leftrotate 1  
state[2] := state[2] leftrotate 2  
state[3] := state[3] leftrotate 3
```

```
return state
```



No se desplaza



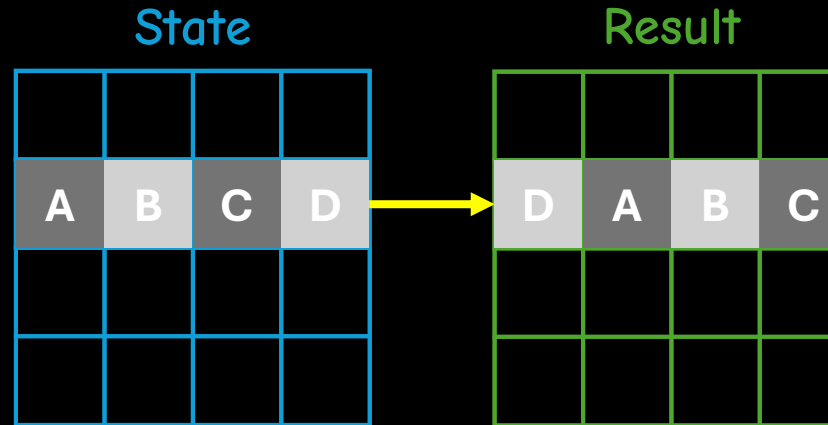
Cifrado en bloque

AES

Shift Rows

Desplaza circularmente las filas del **State**

```
ShiftRows(state[4][4 bytes]):  
    state[0] := state[0] leftrotate 0  
    state[1] := state[1] leftrotate 1  
    state[2] := state[2] leftrotate 2  
    state[3] := state[3] leftrotate 3  
  
    return state
```



Se desplaza 1 byte



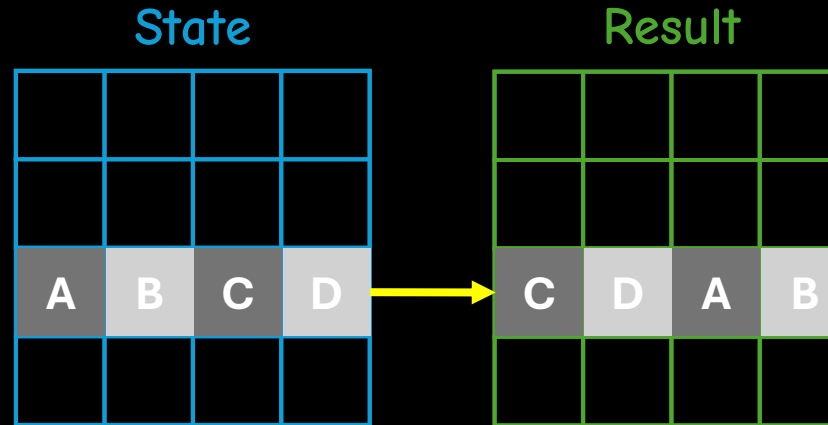
Cifrado en bloque

AES

Shift Rows

Desplaza circularmente las filas del **State**

```
ShiftRows(state[4][4 bytes]):  
    state[0] := state[0] leftrotate 0  
    state[1] := state[1] leftrotate 1  
    state[2] := state[2] leftrotate 2  
    state[3] := state[3] leftrotate 3  
  
    return state
```



Se desplaza 2 bytes



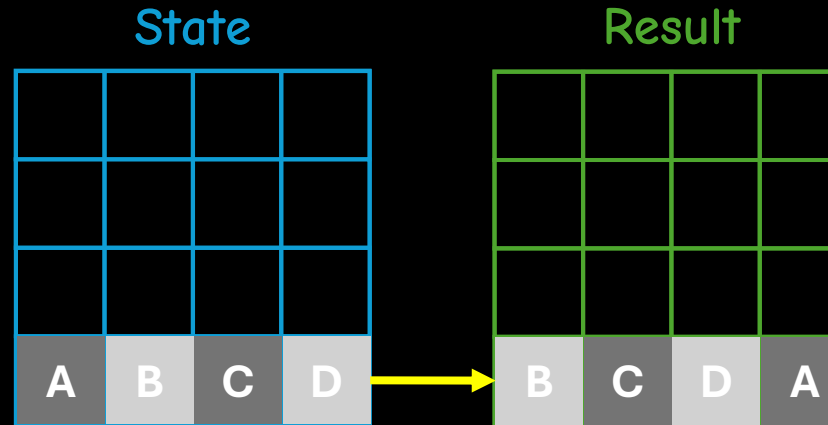
Cifrado en bloque

AES

Shift Rows

Desplaza circularmente las filas del **State**

```
ShiftRows(state[4][4 bytes]):  
    state[0] := state[0] leftrotate 0  
    state[1] := state[1] leftrotate 1  
    state[2] := state[2] leftrotate 2  
    state[3] := state[3] leftrotate 3  
  
    return state
```

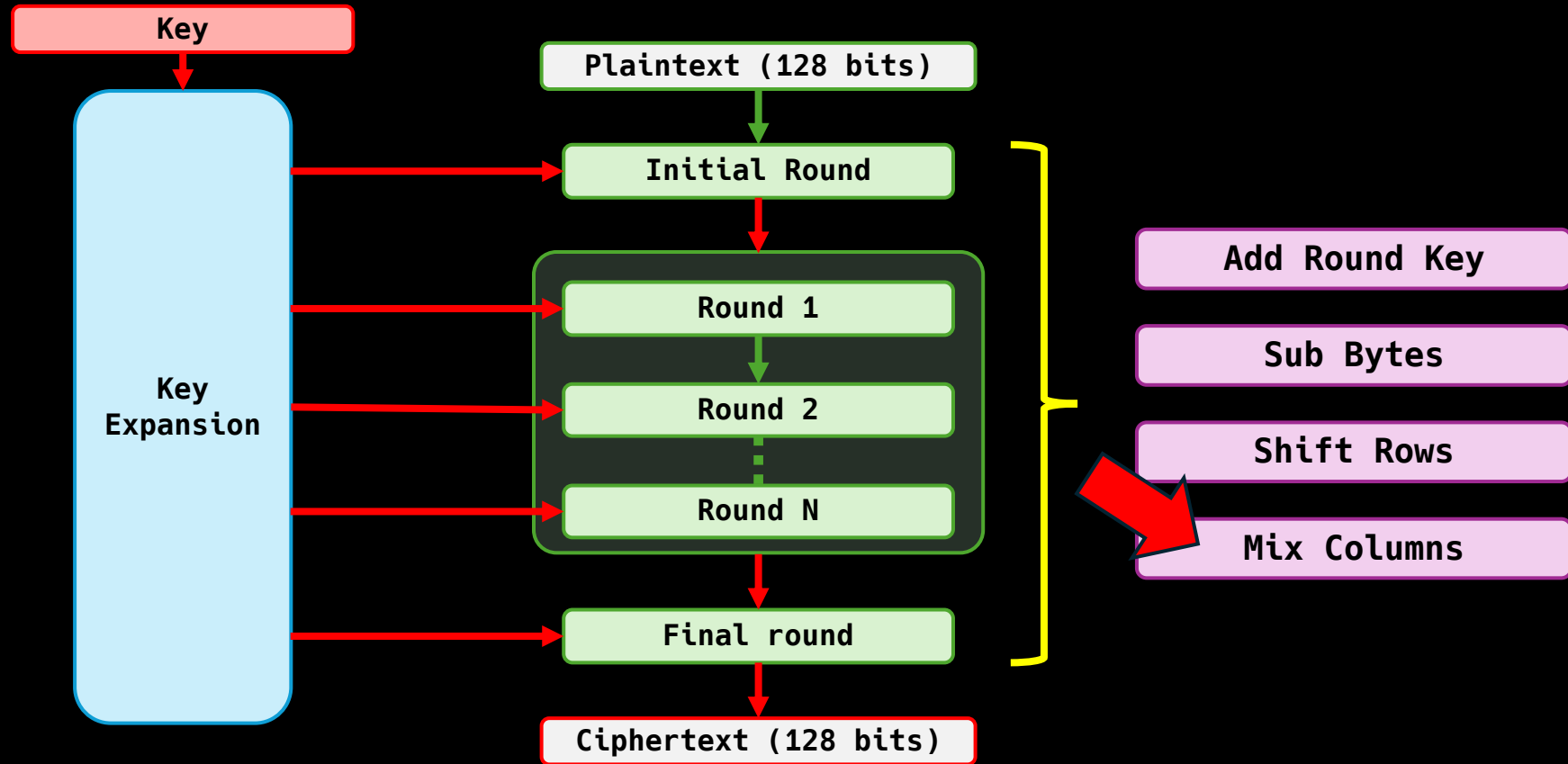


Se desplaza 3 bytes



Cifrado en bloque

AES

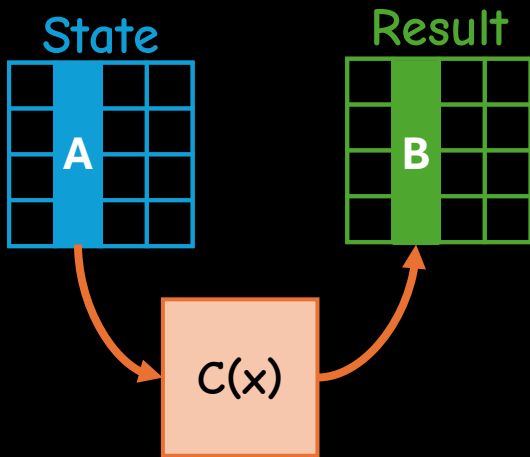


Cifrado en bloque

AES

Mix Columns

Cada columna se interpreta como un vector de 4 bytes y se multiplica por una matriz fija en $GF(2^8)$.

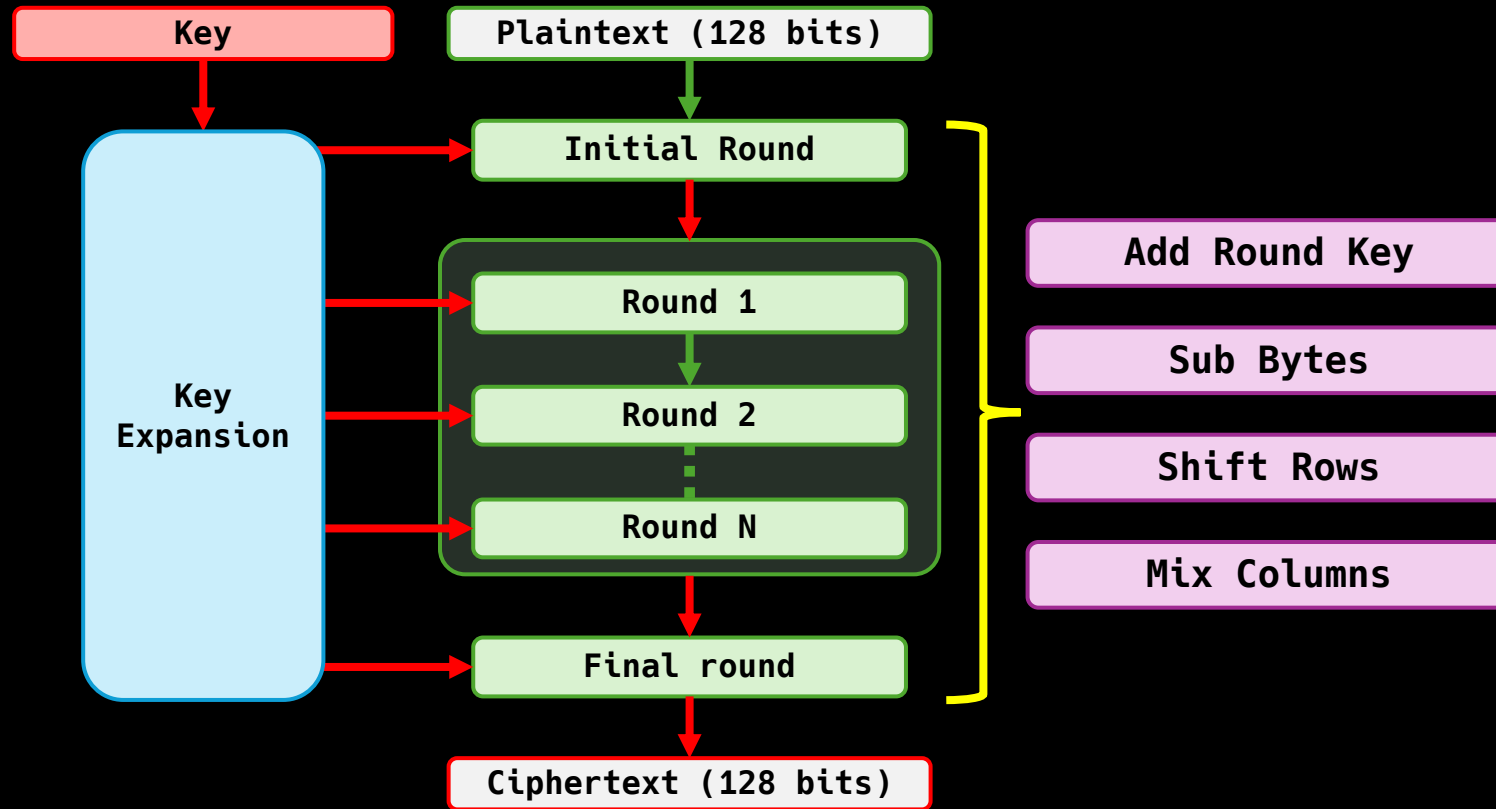


```
MixColumns(state[4][4 bytes]):  
  for column from 0 to 3 do  
    a0 := state[0][column]  
    a1 := state[1][column]  
    a2 := state[2][column]  
    a3 := state[3][column]  
  
    state[0][column] := GMul(0x02, a0) xor GMul(0x03, a1) xor a2 xor a3  
    state[1][column] := a0 xor GMul(0x02, a1) xor GMul(0x03, a2) xor a3  
    state[2][column] := a0 xor a1 xor GMul(0x02, a2) xor GMul(0x03, a3)  
    state[3][column] := GMul(0x03, a0) xor a1 xor a2 xor GMul(0x02, a3)  
  endfor  
  
  return state
```



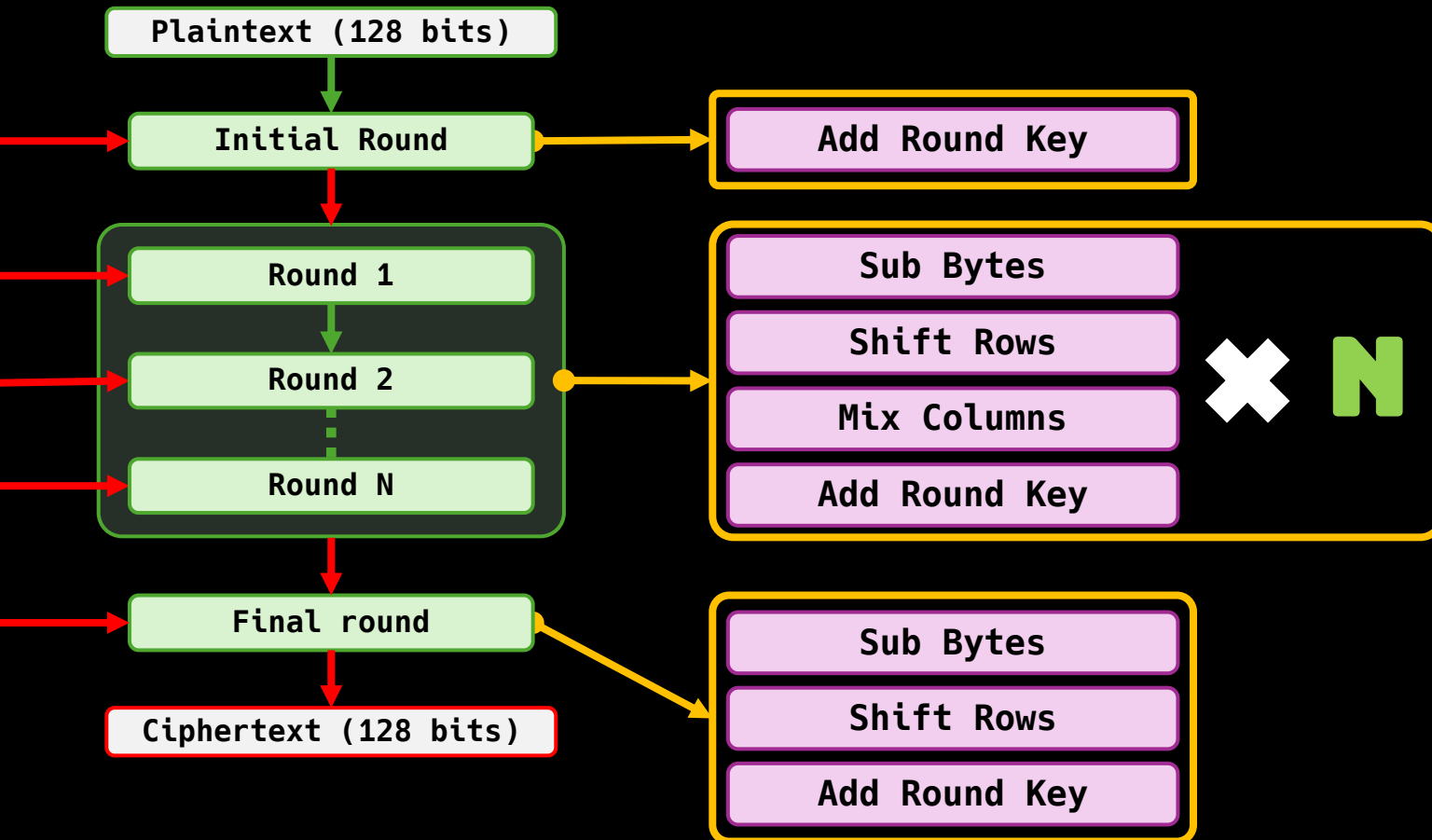
Cifrado en bloque

AES



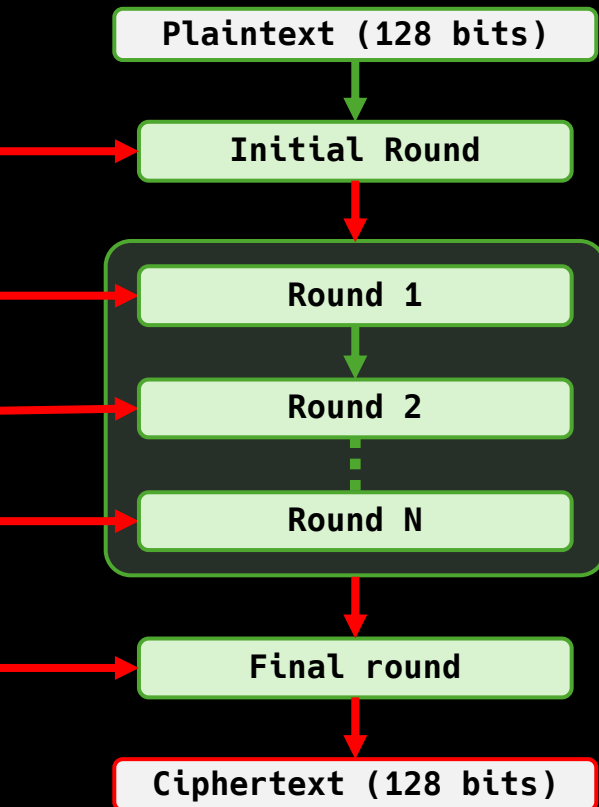
Cifrado en bloque

AES



Cifrado en bloque

AES



```
AESBlockEncrypt(block[128 bits], w, Nr):  
    // 1. Convertir el bloque de entrada en State.  
    state := BytesToState(block)  
  
    // 2. Aplicar AddRoundKey inicial.  
    state := AddRoundKey(state, w, 0)  
  
    // 3. Aplicar las rondas principales.  
    for round from 1 to Nr - 1 do  
        state := SubBytes(state)  
        state := ShiftRows(state)  
        state := MixColumns(state)  
        state := AddRoundKey(state, w, round)  
    endfor  
  
    // 4. Aplicar la ronda final.  
    state := SubBytes(state)  
    state := ShiftRows(state)  
    state := AddRoundKey(state, w, Nr)  
  
    // 5. Convertir el State en bloque de salida.  
    outputBlock := StateToBytes(state)  
  
    return outputBlock
```



Cifrado en bloque

DES



3DES



AES



DrovoLeaks

Public

Unpin

Watch 1

Fork 2

Star 28

main

1 Branch

0 Tags

Go to file

Add file



drovoh4k	Nueva clase: Metodología análisis de binarios	6a7d86c · 2 weeks ago	42 Commits
cursos	Nueva clase: Metodología análisis de binarios	2 weeks ago	
masterclass	Refactor: meta.json removed + Links corrected + Images c...	2 months ago	
tutoriales	Refactor: meta.json removed + Links corrected + Images c...	2 months ago	
varios/ida_vs_ghidra	Nuevo video: Ida VS Ghidra	4 months ago	
vulnerabilidades	Nueva vulnerabilidad: CVE-2026-3909 y CVE-2026-3910 ...	last month	
writeups	Nuevo CTF: Hack4uCTF (crypto) - Locked Away	last month	
README.md	Nueva masterclass: Introducción a MIPS	3 months ago	

README

DrovoLeaks

Channel @drovoh4k

Este repositorio es el **material de apoyo** de mis vídeos de YouTube.

La idea es tener todo el conocimiento del canal en un lugar centralizado y ordenado, no solo los propios videos, sino también todos los recursos extra.

Navegación rápida

- [Writeups](#)
 - Resoluciones de retos (crackmes, plataformas y CTFs), con scripts y recursos.
- [Cursos](#)
 - Contenido estructurado en módulos y clases: enfoque progresivo, con enlaces a playlist y recursos.
- [Masterclass](#)

About

No description, we provided.

- Readme
- Activity
- 28 stars
- 1 watching
- 2 forks

Releases

No releases published
[Create a new release](#)

Contributors 1

drovoh4k Llor

Languages

- Python 76.4%

C 6.2%

Shell 3.5%

Doc

Suggested workflows

Based on your tech stack

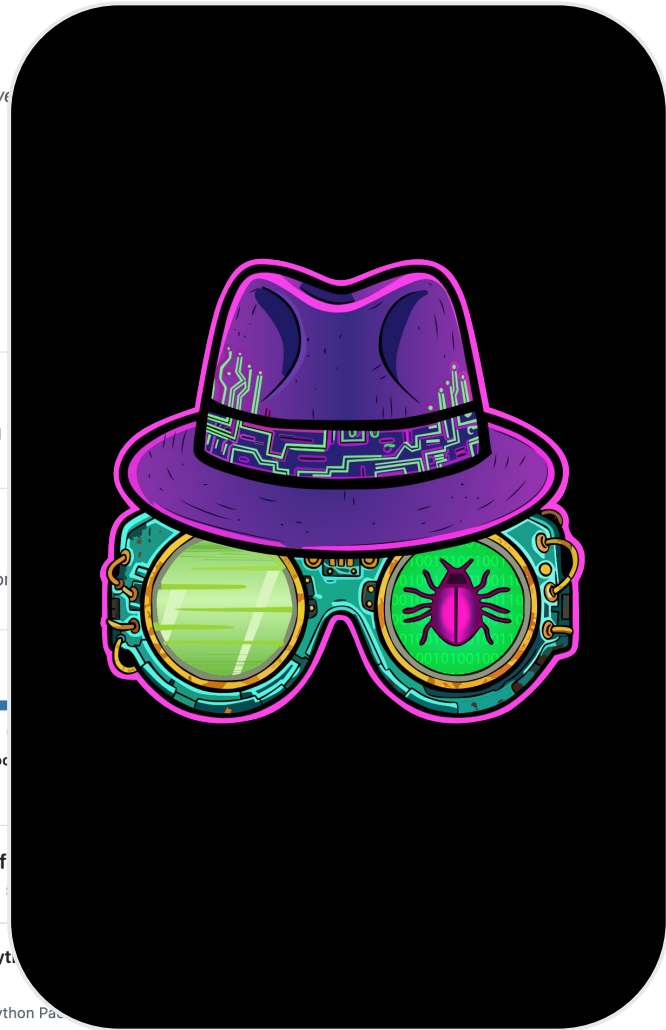
Publish Python Package

Publish a Python Package on release.

Python application

Create and test a Python application.

Configure



tutoriales	Refactor: meta.json removed + Links corrected + Images c...	2 months ago
varios/ida_vs_ghidra	Nuevo video: Ida VS Ghidra	4 months ago
vulnerabilidades	Nueva vulnerabilidad: CVE-2026-3909 y CVE-2026-3910 ...	last month
writeups	Nuevo CTF: Hack4uCTF (crypto) - Locked Away	last month
README.md	Nueva masterclass: Introducción a MIPS	3 months ago

README

DrovoLeaks

Channel @drovoh4k

Este repositorio es el **material de apoyo** de mis vídeos de YouTube.

La idea es tener todo el conocimiento del canal en un lugar centralizado y ordenado, no solo los propios videos, sino también todos los recursos extra.

Navegación rápida

- [Writeups](#)
 - Resoluciones de retos (crackmes, plataformas y CTFs), con scripts y recursos.
- [Cursos](#)
 - Contenido estructurado en módulos y clases: enfoque progresivo, con enlaces a playlist y recursos.
- [Masterclass](#)
 - Clases específicas para quienes ya dominan los fundamentos: se asume base previa y se entra directamente en una temática concreta con enfoque técnico y práctico.
- [Tutoriales](#)
 - Guías/tips independientes sobre conceptos o tooling.
- [Vulnerabilidades](#)
 - CVEs concretos: análisis técnico de la falla, impacto, condiciones de explotación y una PoC guiada paso a paso.

! Disclaimer

Contenido con fines educativos. Respeta las normas de cada plataforma y licencias de terceros.

1 watching

2 forks

Releases

No releases published
[Create a new release](#)

Contributors 1

 **drovoh4k** Llor

Languages

Python 76.4%
C 6.2%
Shell 3.5%

Suggested workflows

Based on your tech stack

 **Publish Python Package**
Publish a Python package on release.

 **Python application**
Create and test a Python application.

 **Django**
Build and Test Django application.

[More workflows](#)



